CPU Pipelining

CS204

By

Dr. Sk Subidh Ali

Car Industry Pipeline Processing



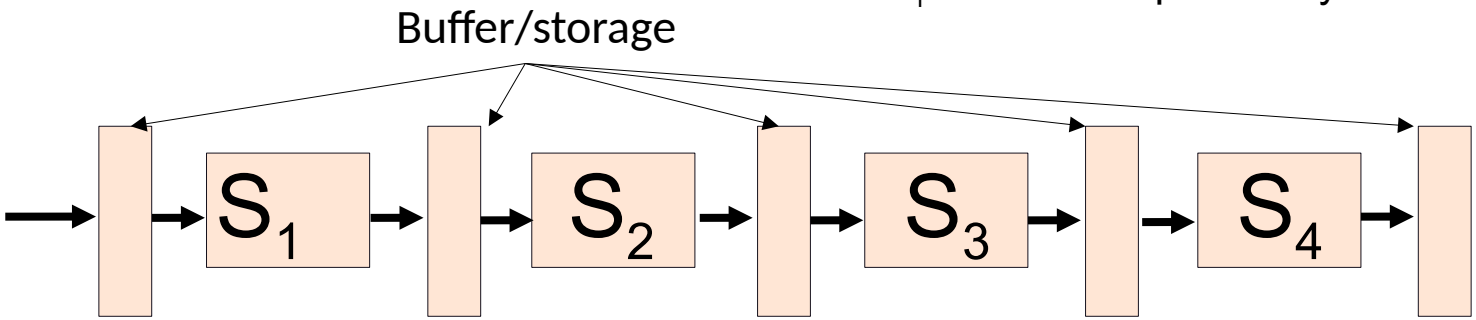
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



Block B_i can execute T_i in one cycle

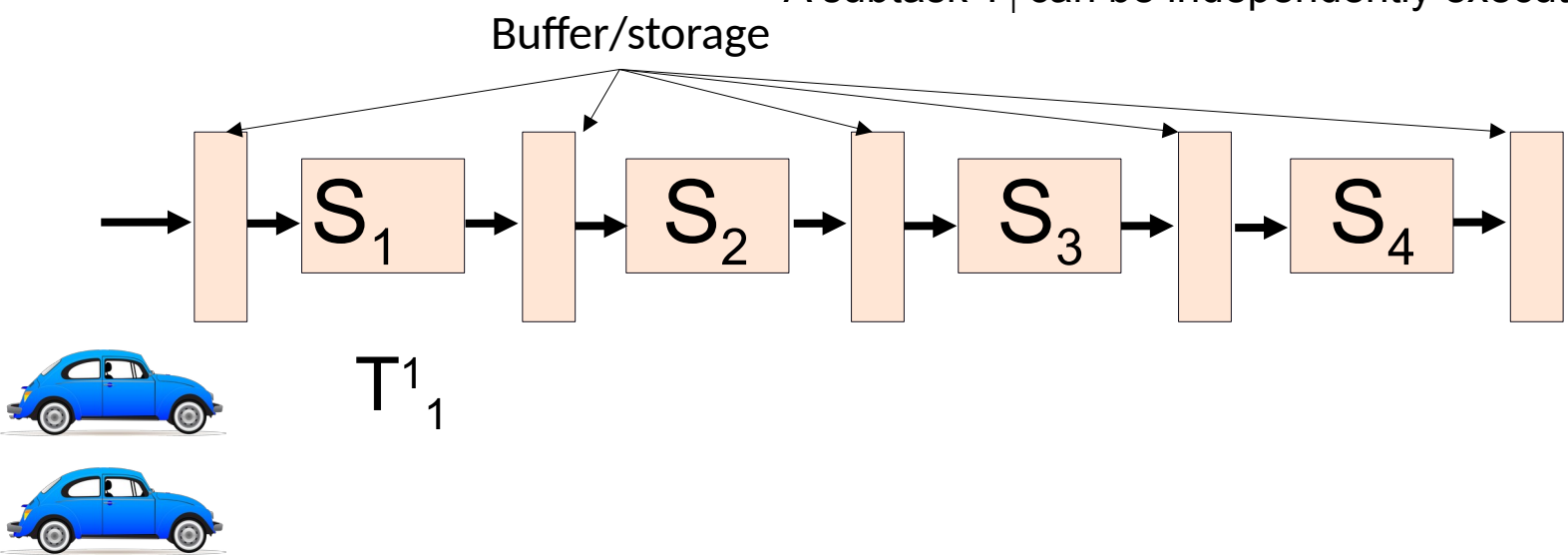
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



CLK$_1$

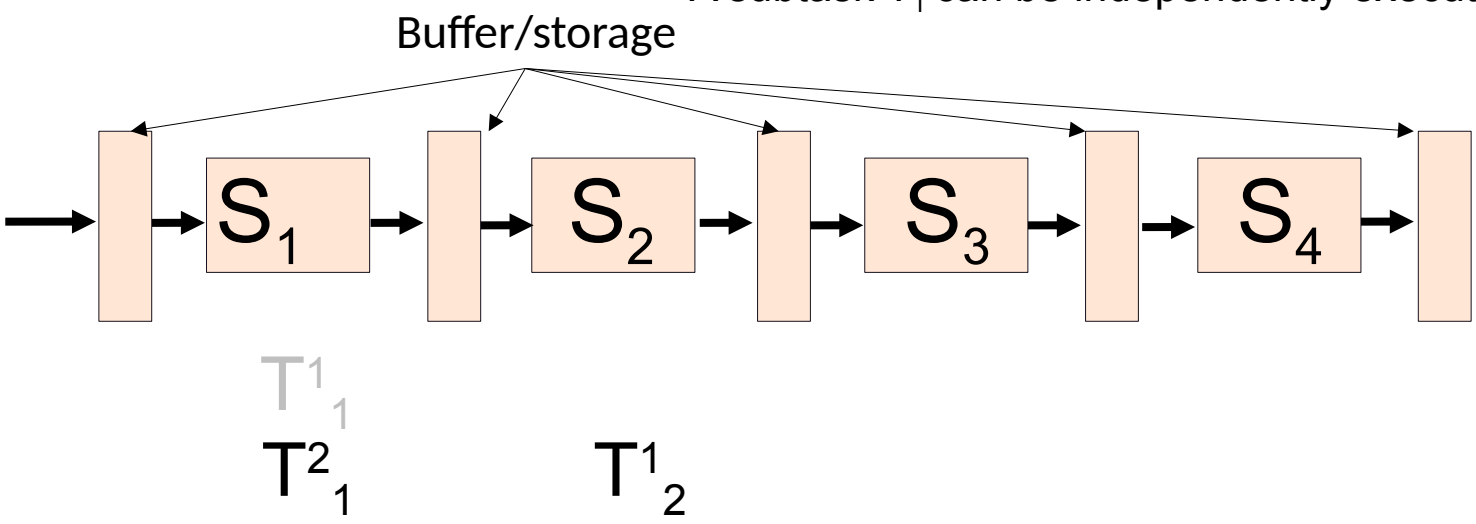
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



CLK_2

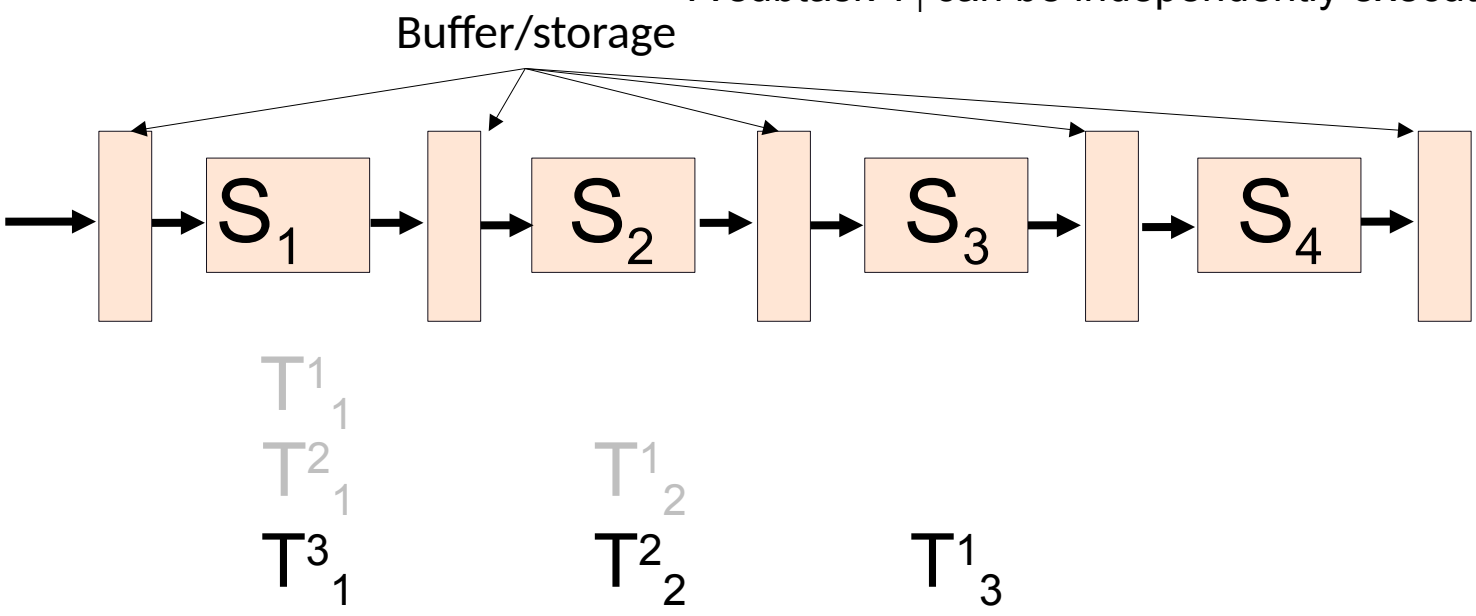
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



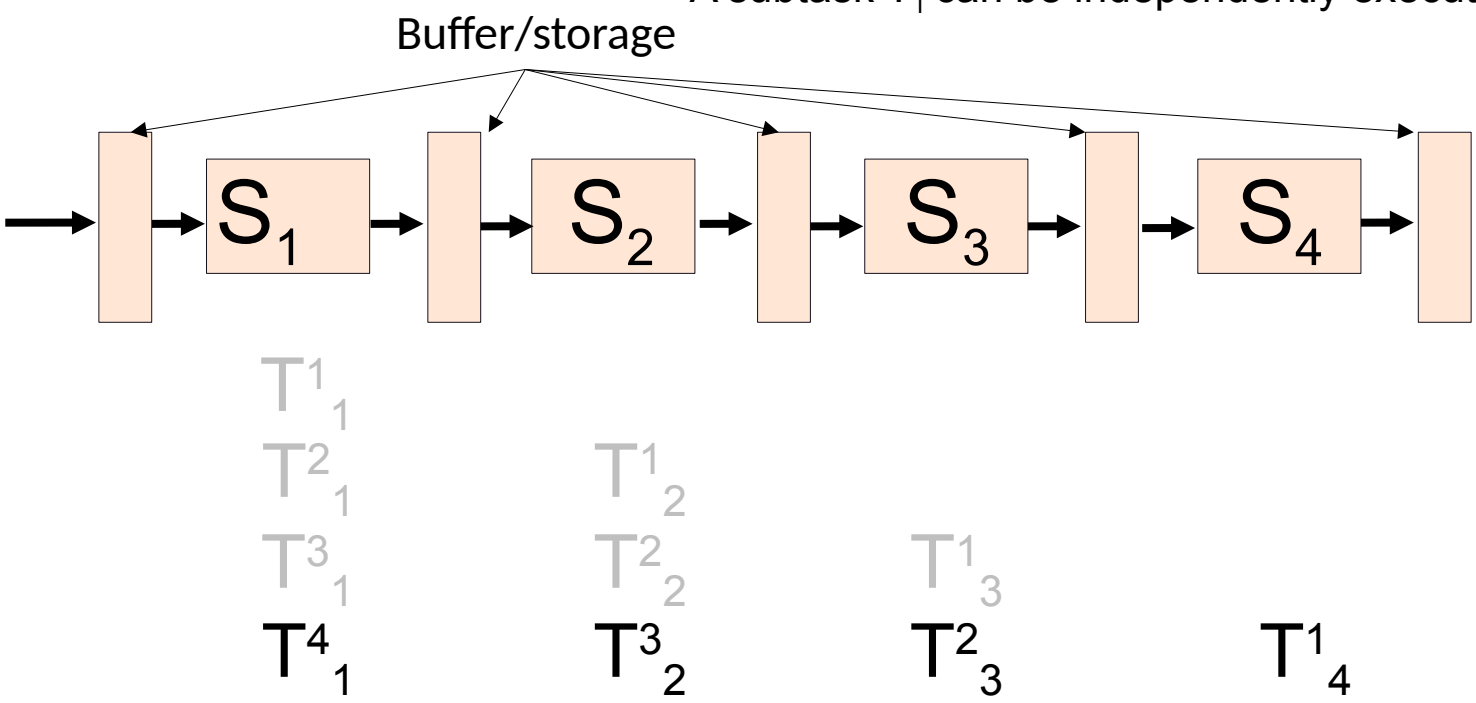
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



The pipeline is full: each stage is busy

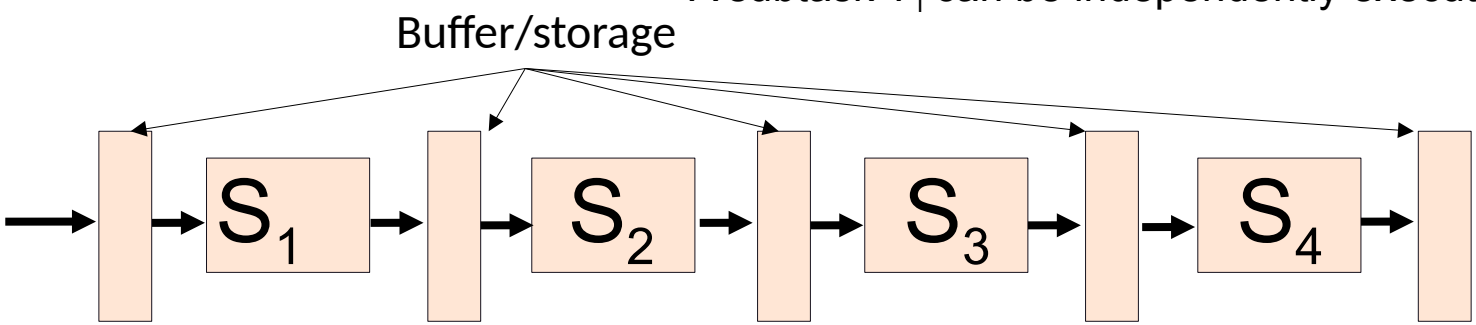
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



T^5_1 T^4_2 T^3_3 T^2_4

CLK_5

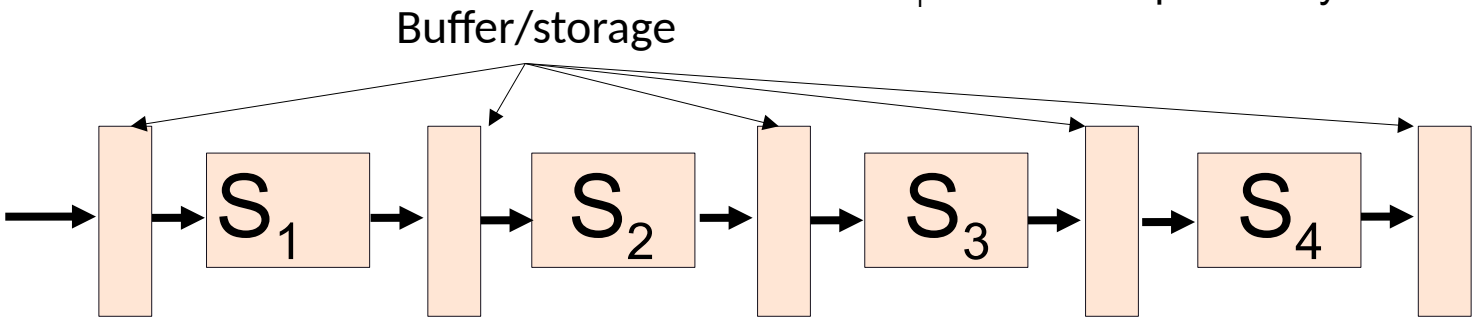
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



How many cycles required to complete n tasks?

T^6_1 T^5_2 T^4_3 T^3_4

CLK_6



CPU Instruction Pipelining

Case studies:

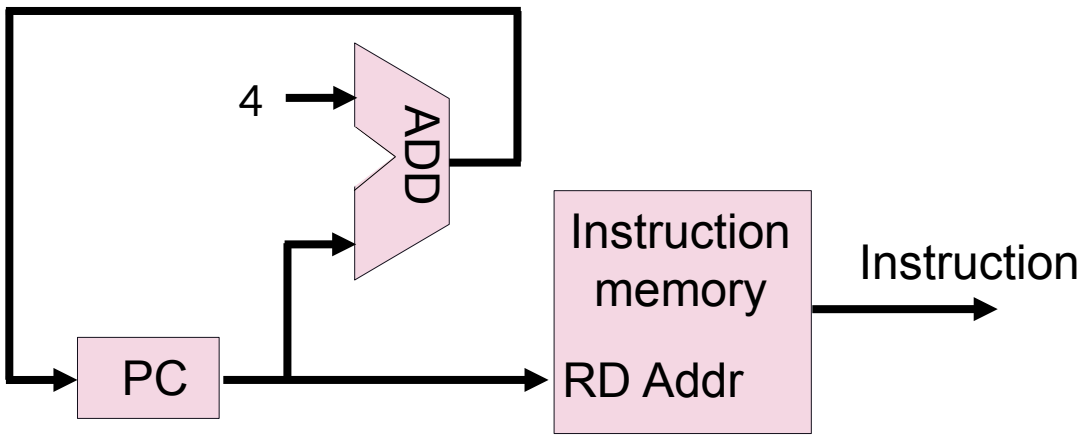
- Register referencing: ADD R_1, R_2, R_3 => R_1=R_2 + R_3
- Memory referencing:

  Load R_1, n(R_2) => R_1= [R_2+ n] // n is a constant number

  Store R_1, n(R_2) => [R_2+ n] = R_1 // n is a constant number
- Jump: JE R_1, R_2, L => Jump to L if R_1=R_2 // L is offset value

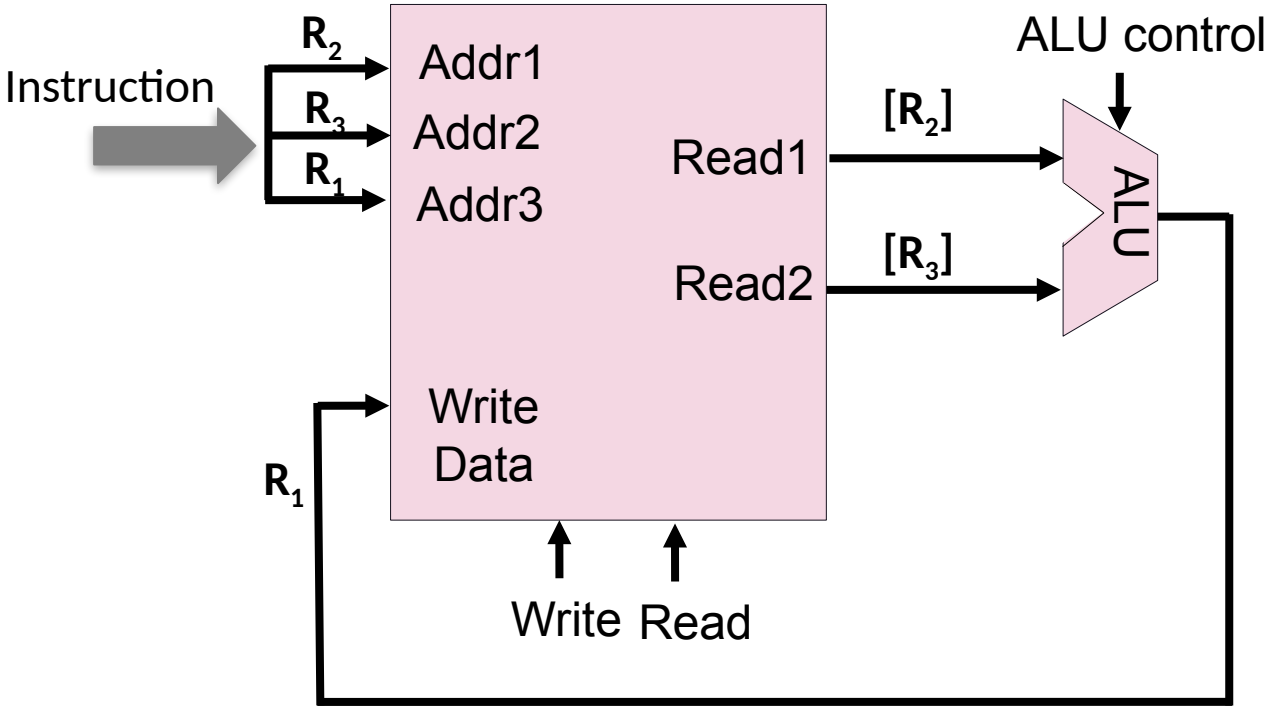
Consider each instruction is of 4 bytes or 32 bits

Instruction fetch (IF)



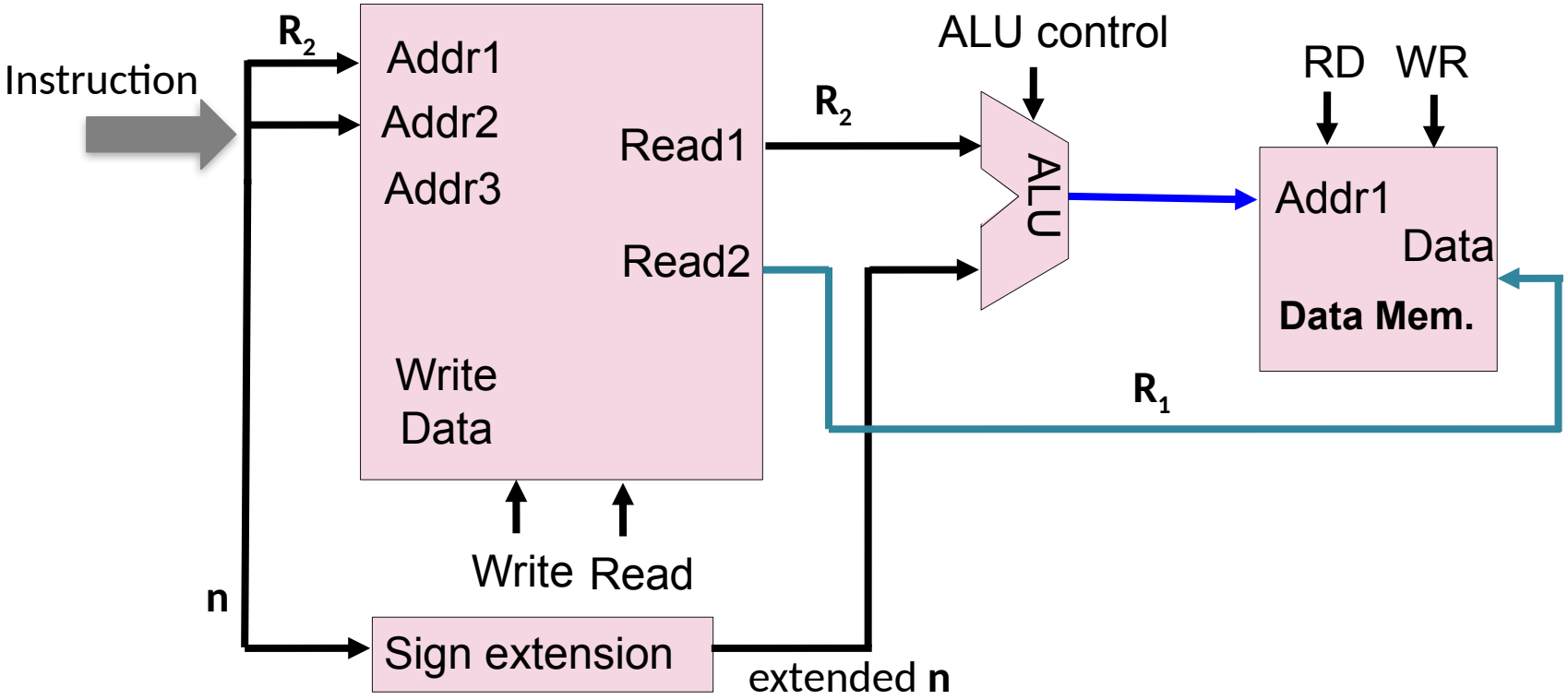
CPU Instruction Pipelining

- Register referencing: **ADD R_1, R_2, R_3**



CPU Instruction Pipelining

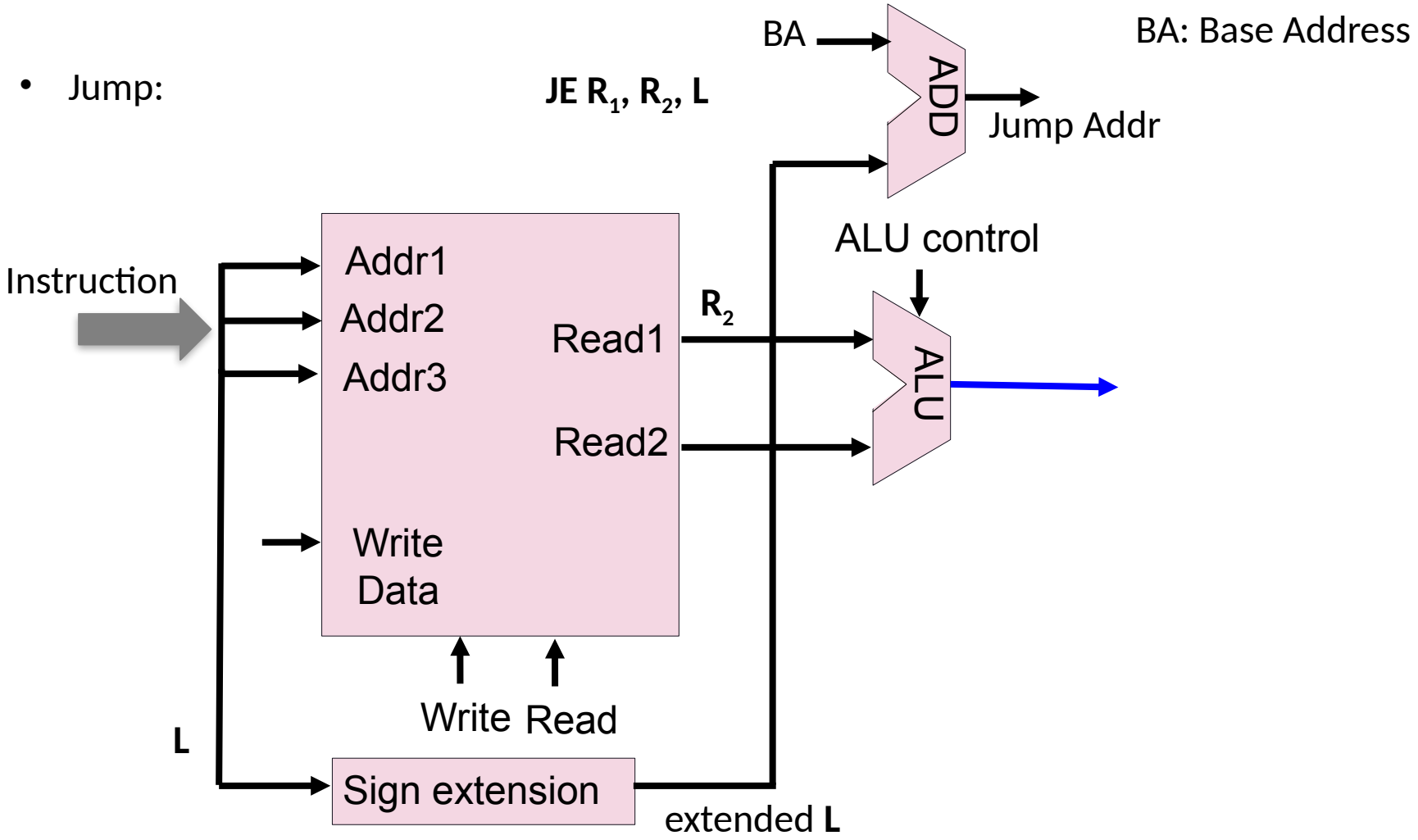
- Memory referencing: **Store R_1, n(R_2)**



CPU Instruction Pipelining

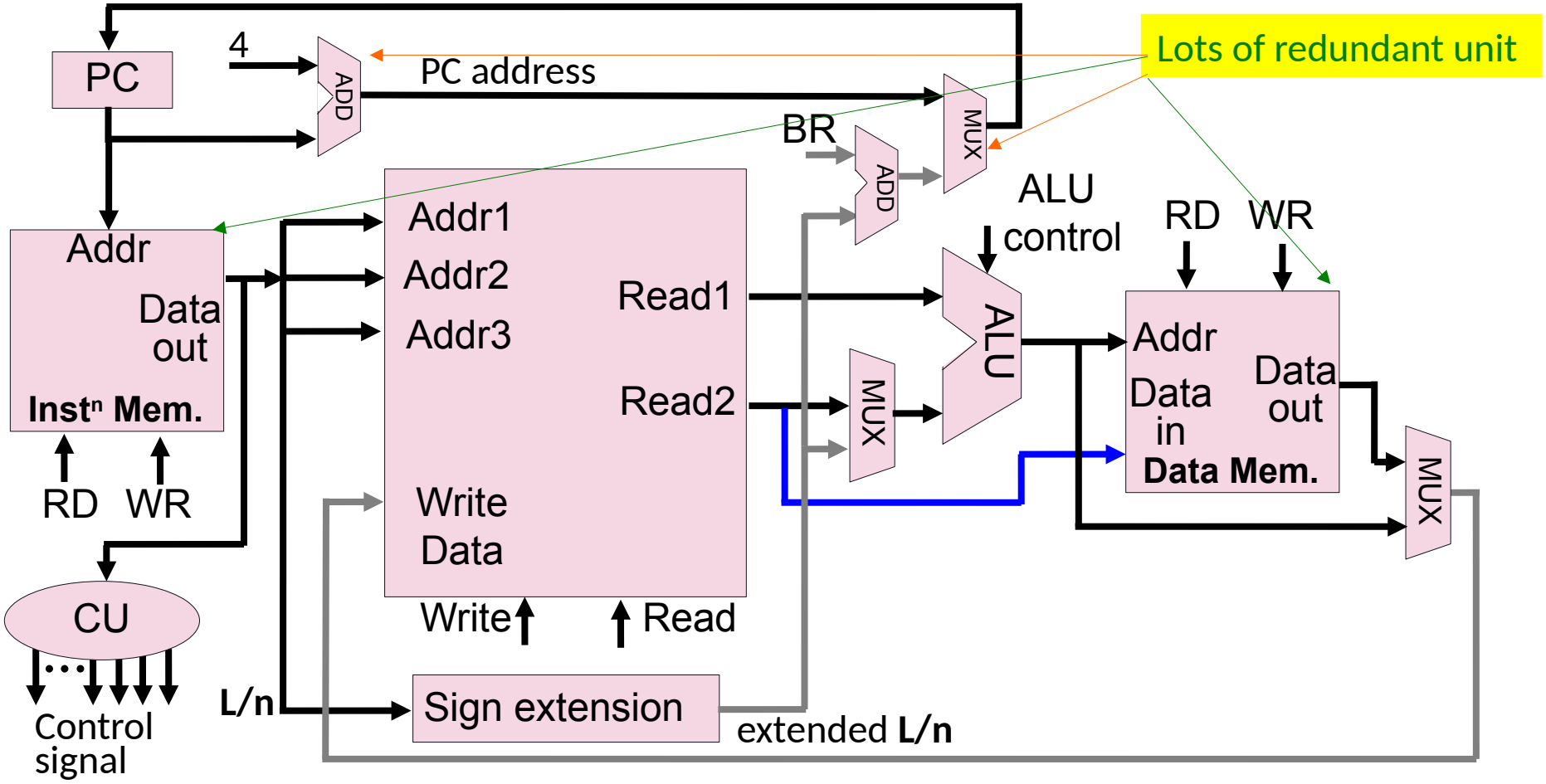
- Jump:

JE R_1, R_2, L

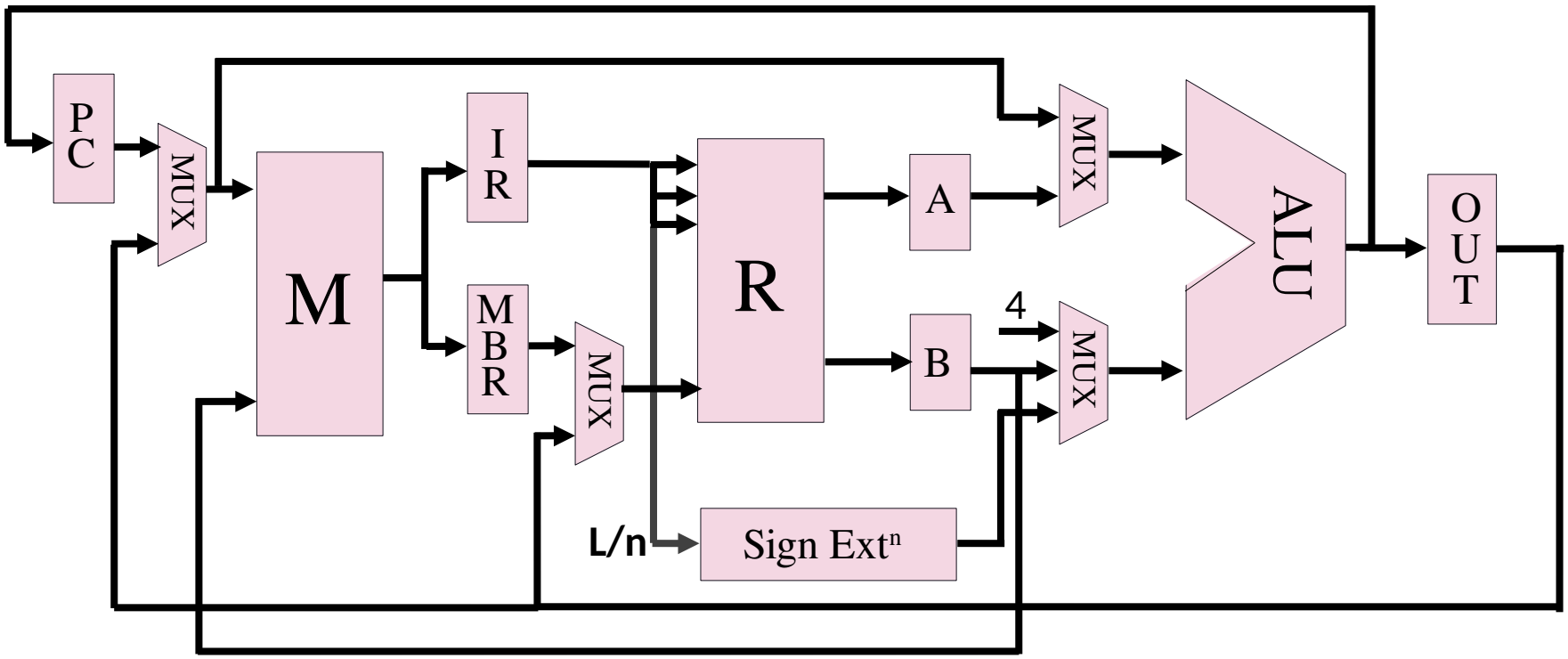


CPU Design

ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**



Optimized Design of CPU



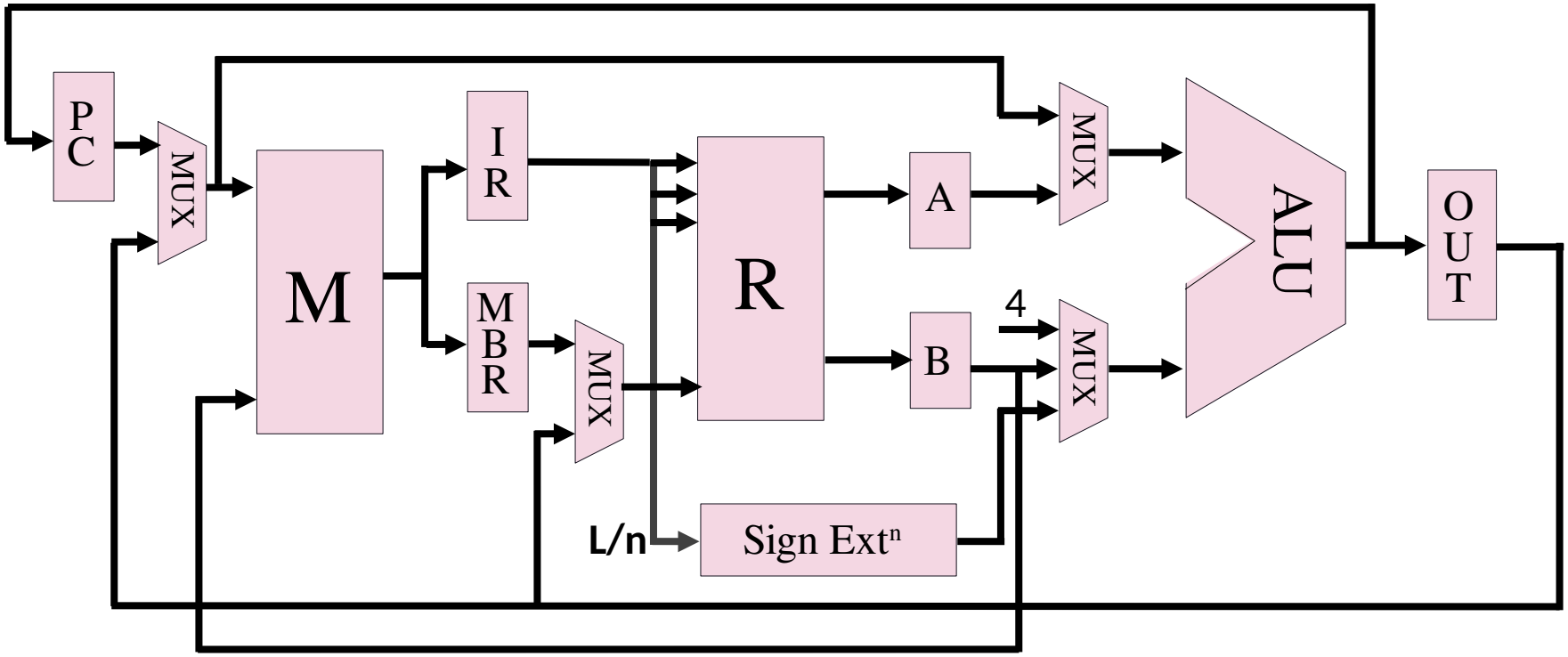
Optimized Design of CPU

Instruction execution

Fetch

IR=M[PC]

PC=PC+4



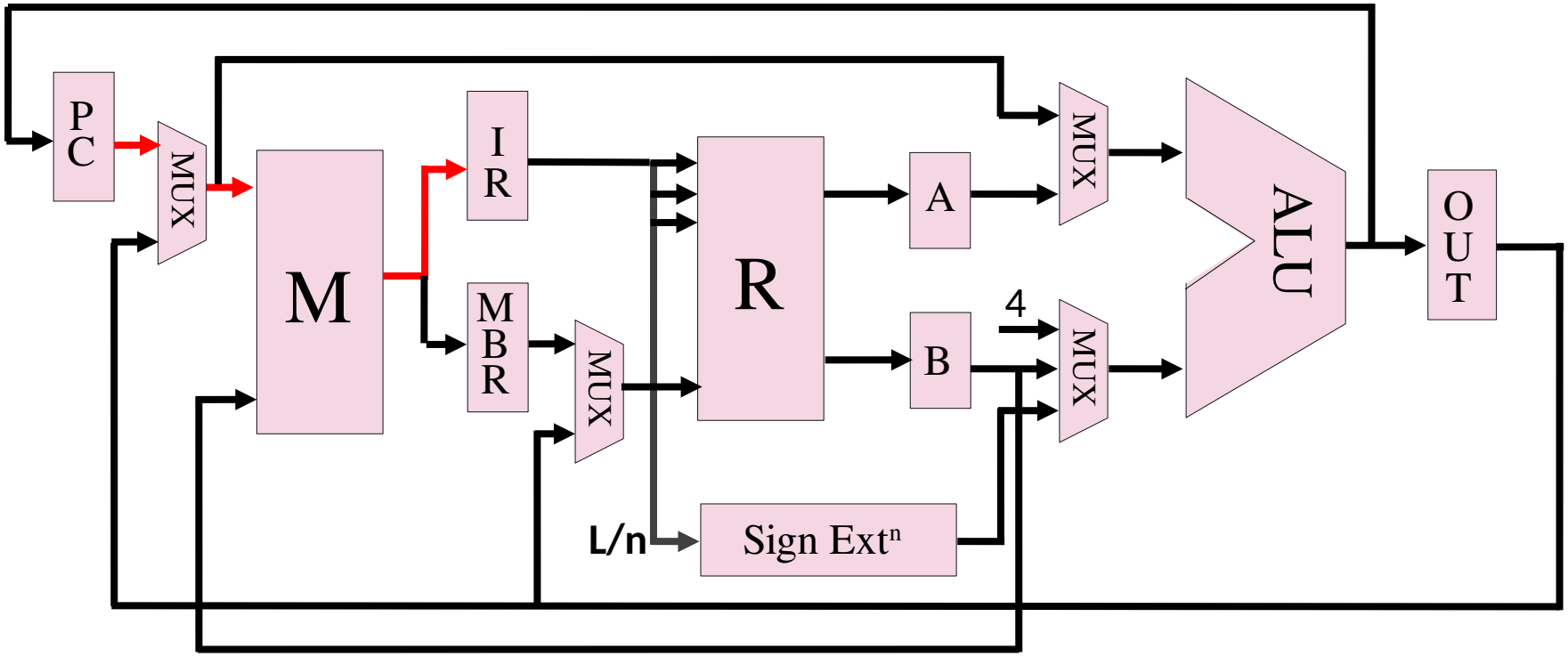
Optimized Design of CPU

Instruction execution

Fetch

IR=M[PC]

PC=PC+4



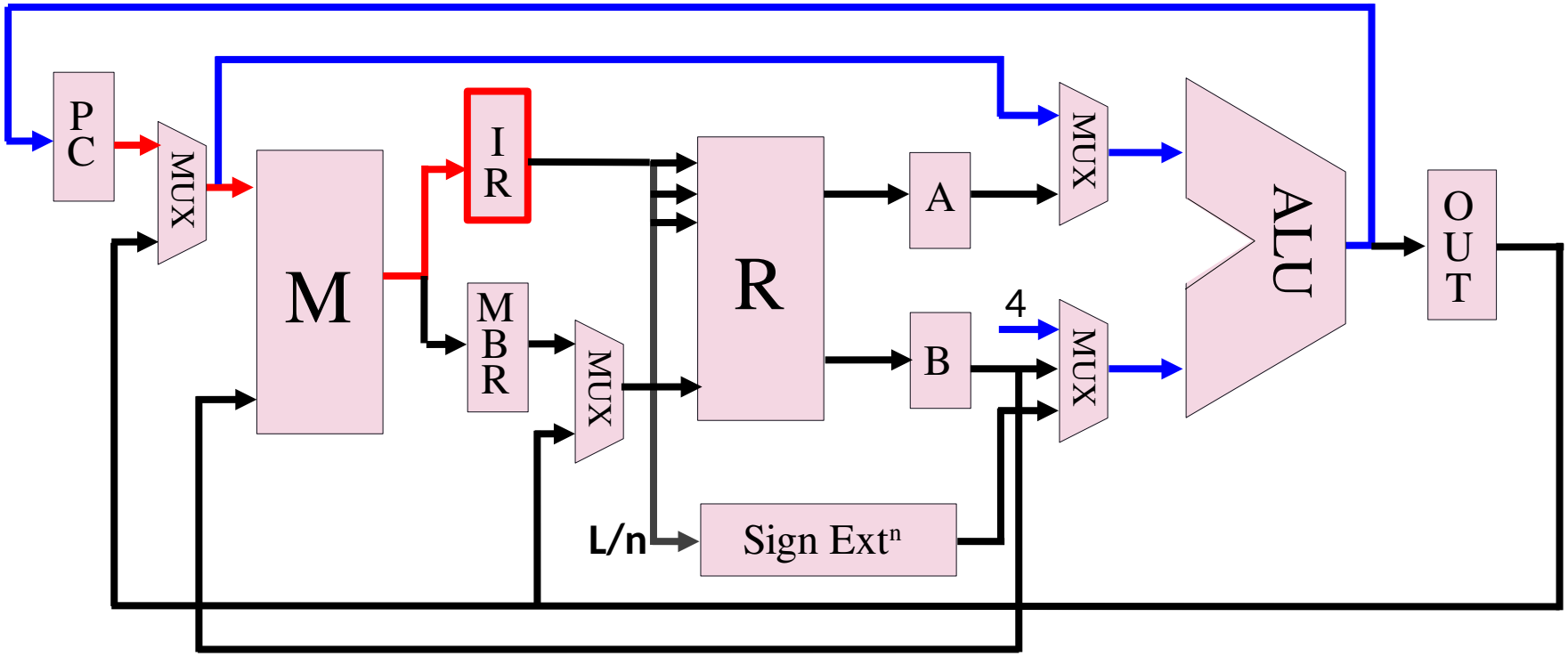
Optimized Design of CPU

Instruction execution

T_0 Fetch

IR=M[PC]

PC=PC+4

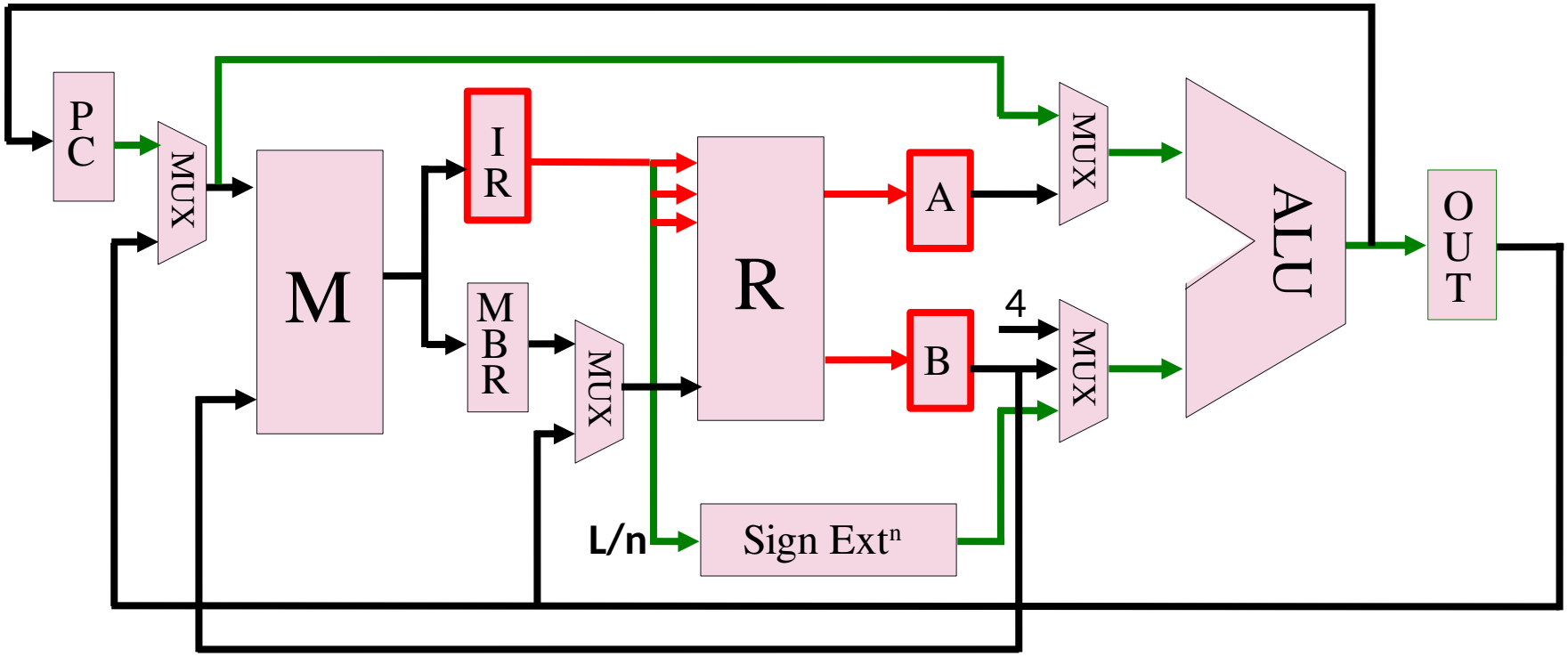


Optimized Design of CPU

Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L



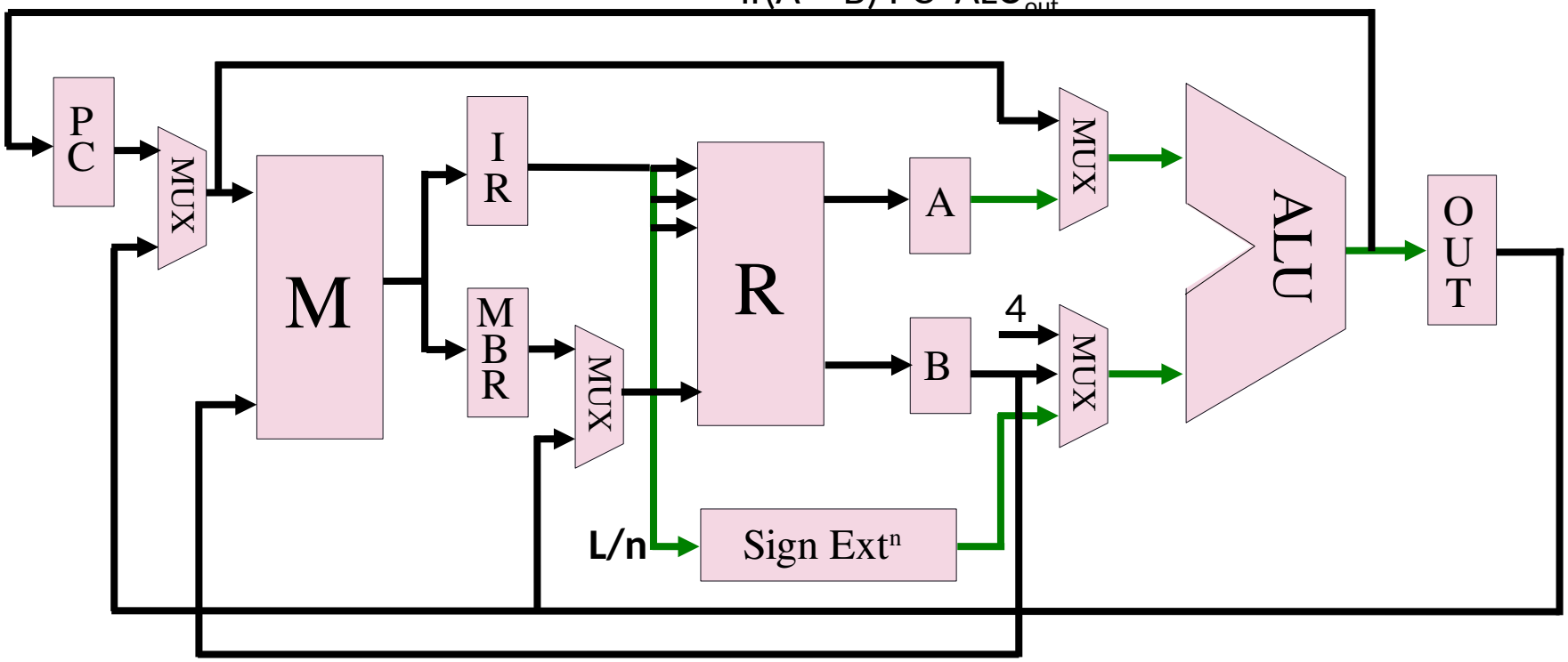
Optimized Design of CPU

Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}



ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**

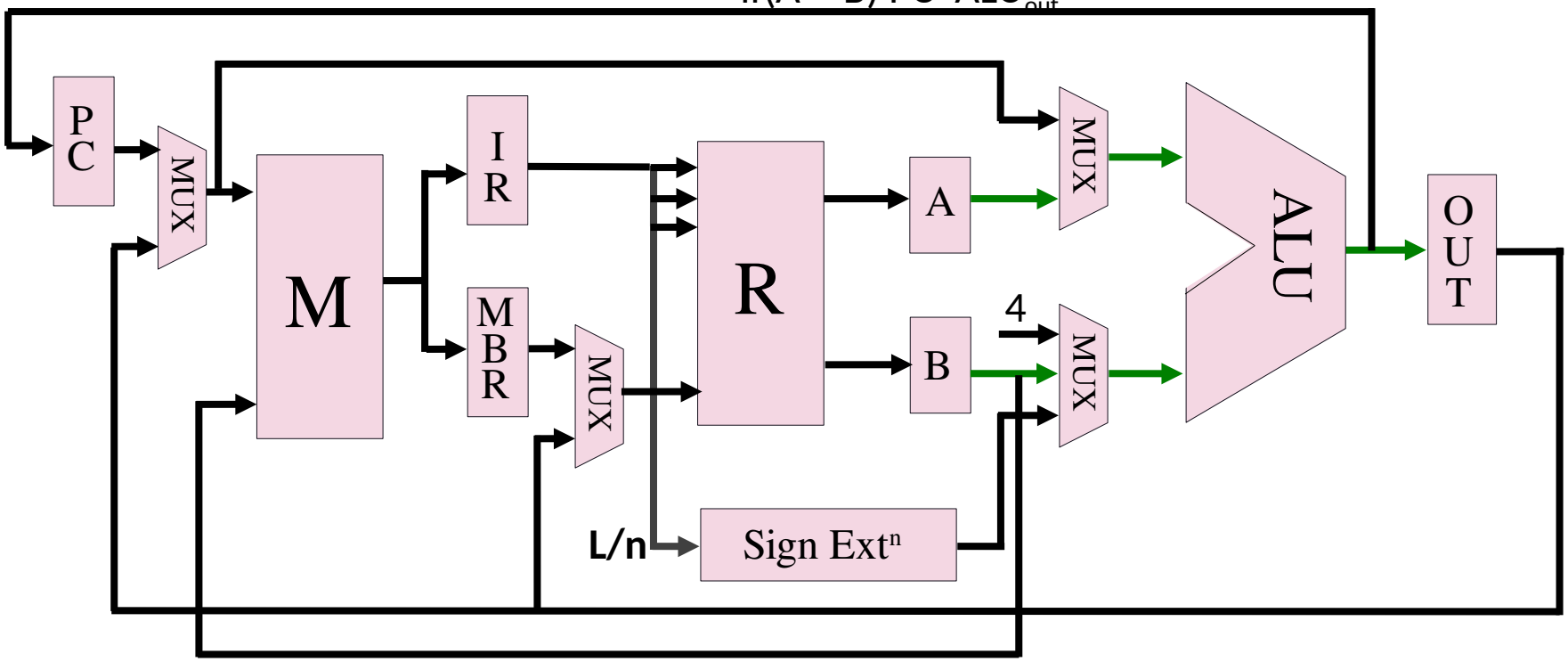
Optimized Design of CPU

Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}



ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**

Optimized Design of CPU

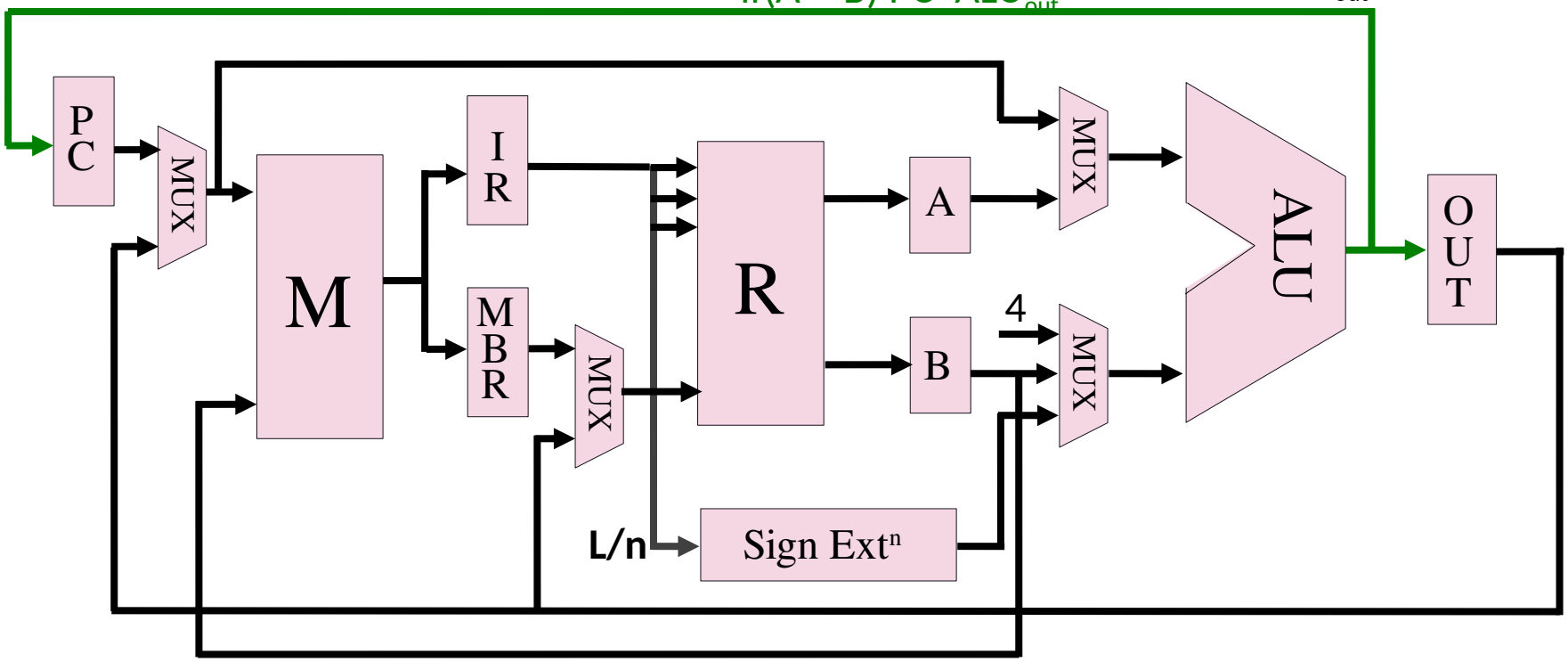
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : R



ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**

Optimized Design of CPU

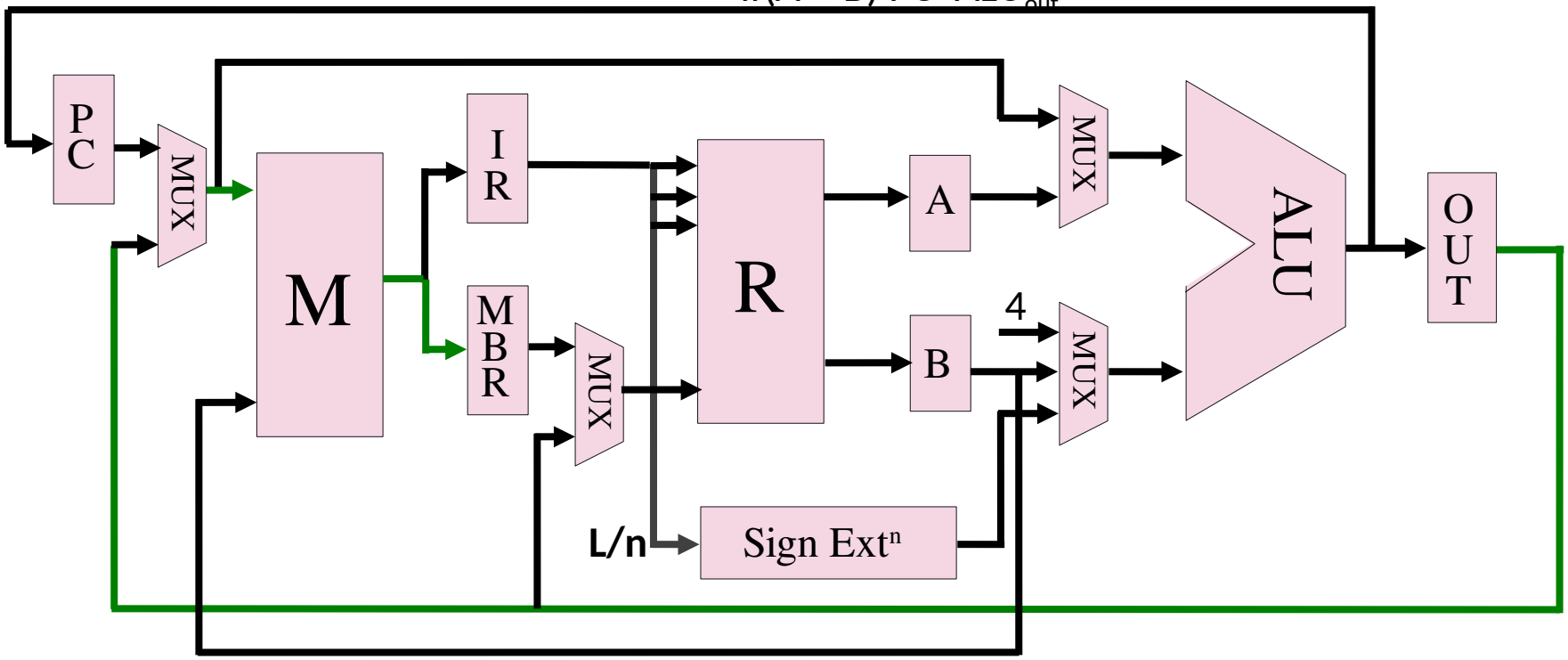
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : R



ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**

Optimized Design of CPU

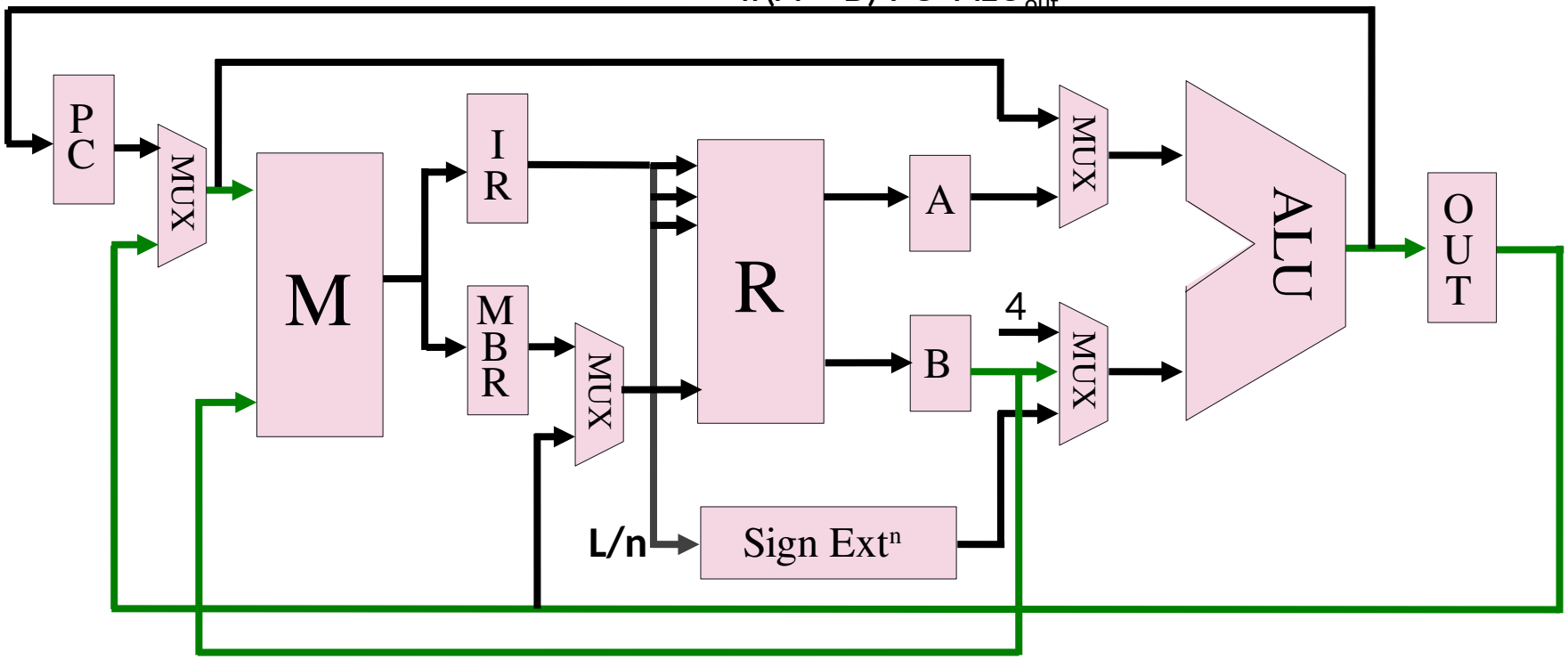
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : R



ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**

Optimized Design of CPU

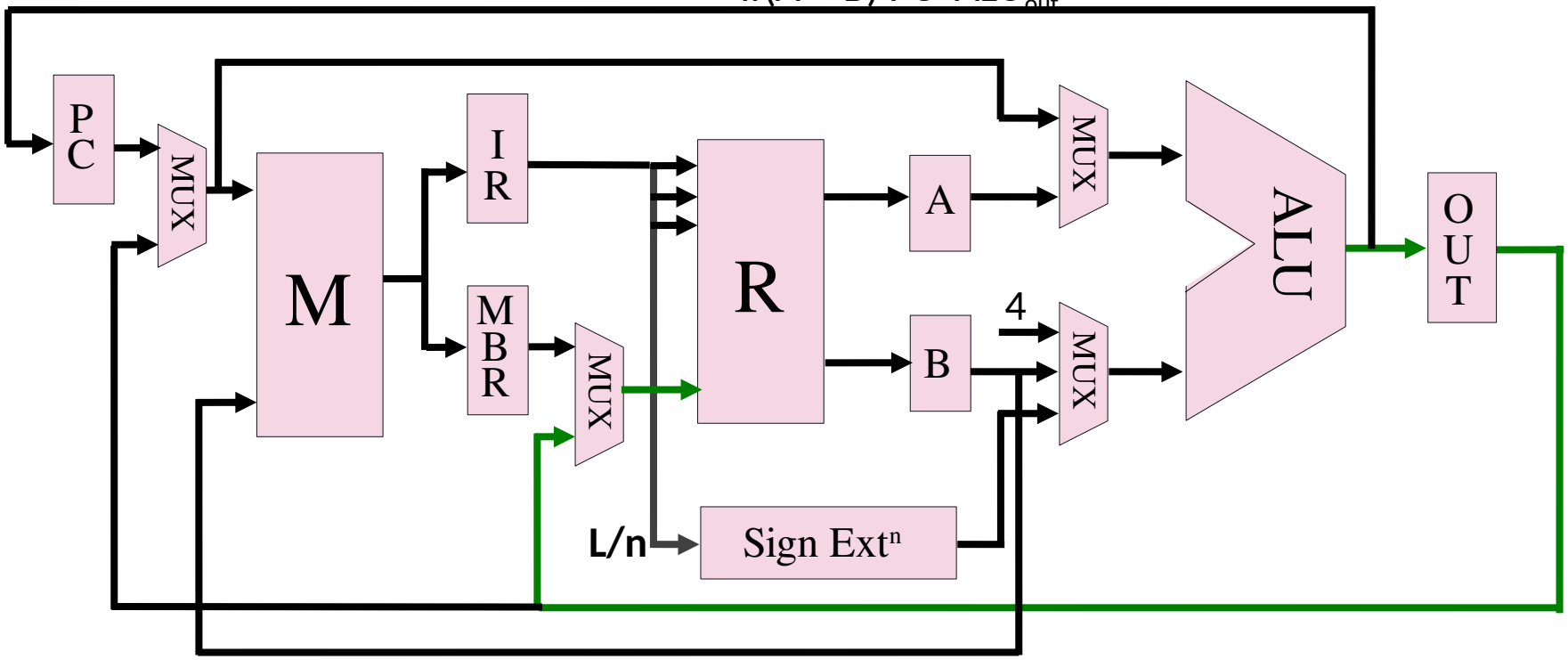
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : R



ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**

Optimized Design of CPU

Instruction execution

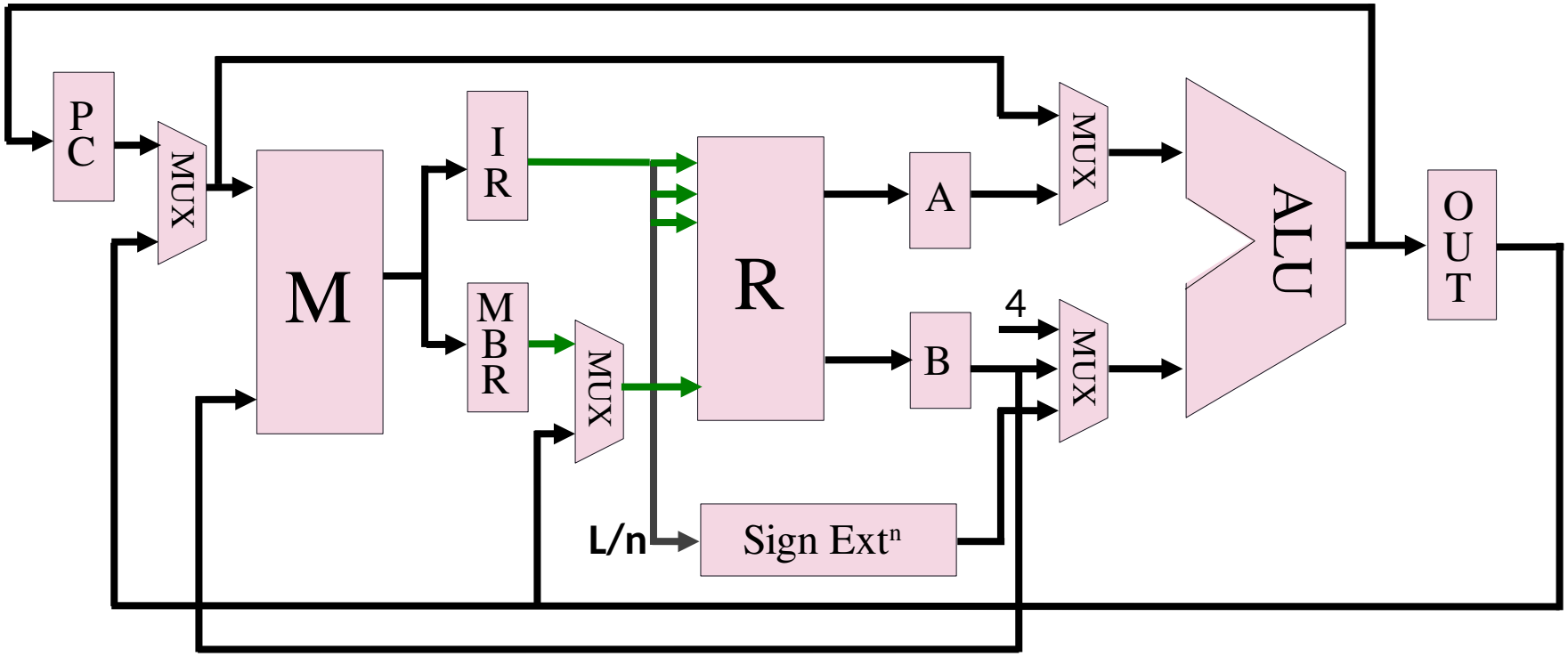
T_5 **LD**

R[IR]=MDR

Clock speed is faster

Hardware cost reduced

Pipelining is difficult



ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**

Optimized Design of CPU

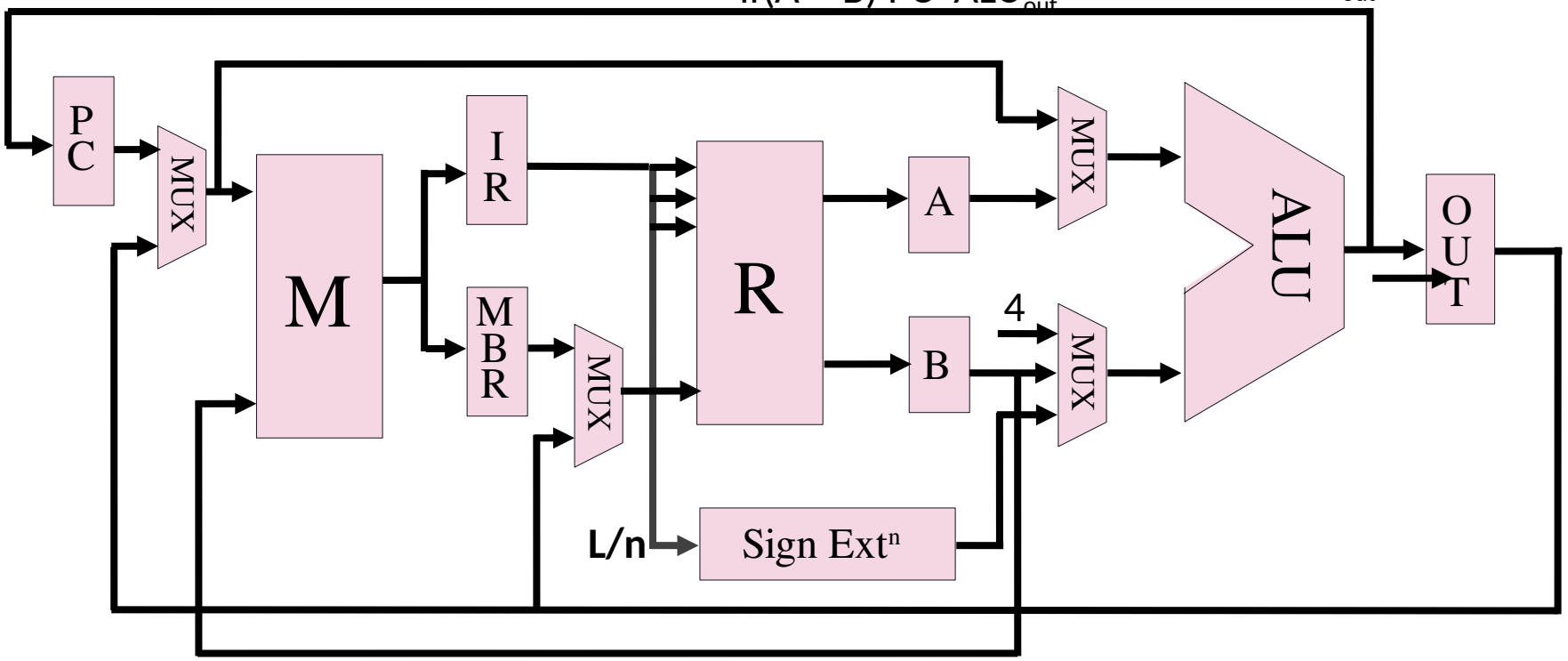
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

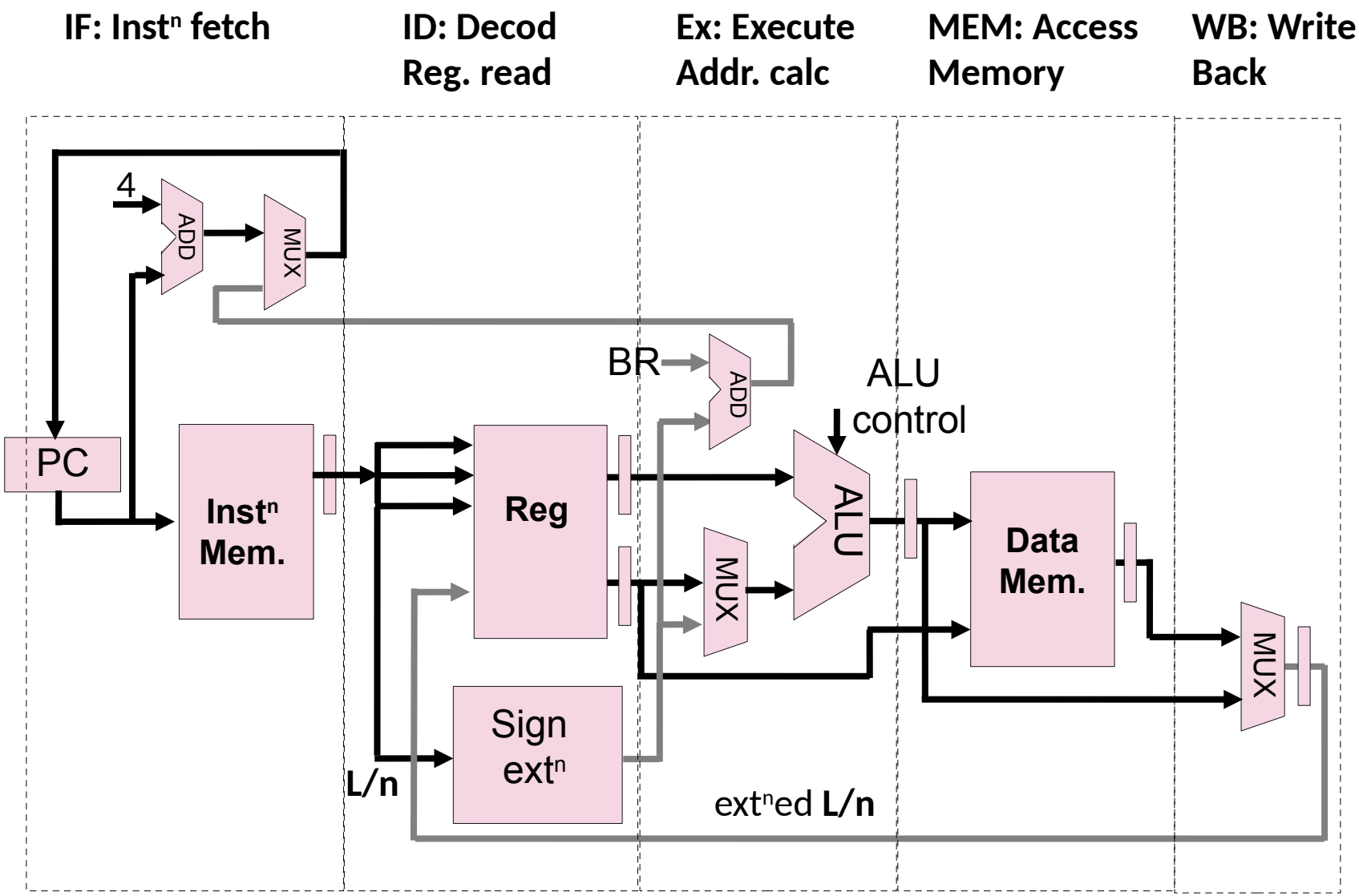
T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : MDR



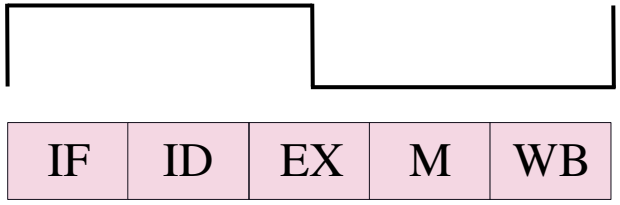
ADD R_1, R_2, R_3 **Store R_1, n(R_2)** **LD R_1, n(R_2)** **JE R_1, R_2, L**

CPU Pipe-lined Design

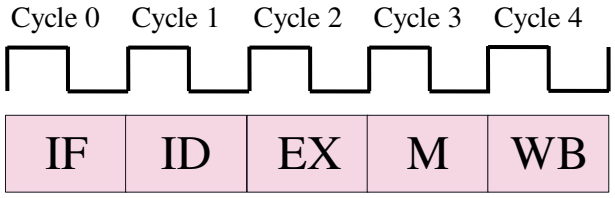


CPU Pipelining

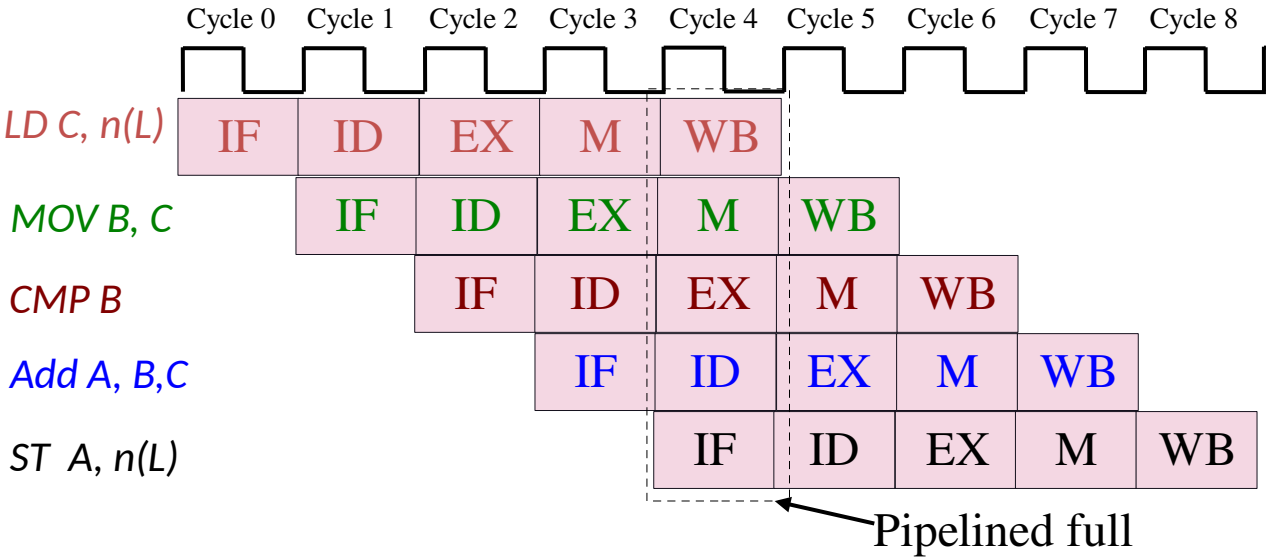
CPU Pipelining



Single-Cycle CPU



Multi-Cycle CPU



Pipelined CPU

Advantages of pipelining ,

- High throughput
- High CPU utilization

Problems with pipelining (Hazards)

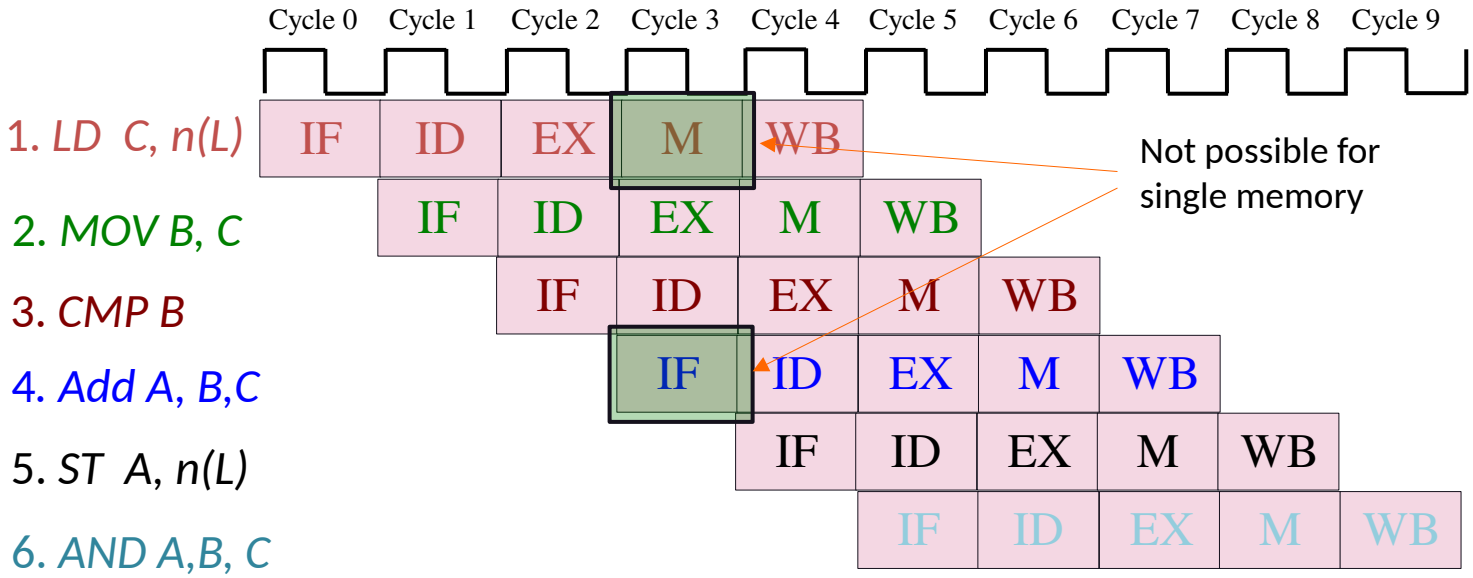
- Structural hazard
- Data hazard
- Control hazard

Problems with pipelining (Hazards)

- Structural hazard
- Data hazard
- Control hazard

CPU Pipelining Hazards

Structural hazard: CPU does not allow two instructions execute at the same time



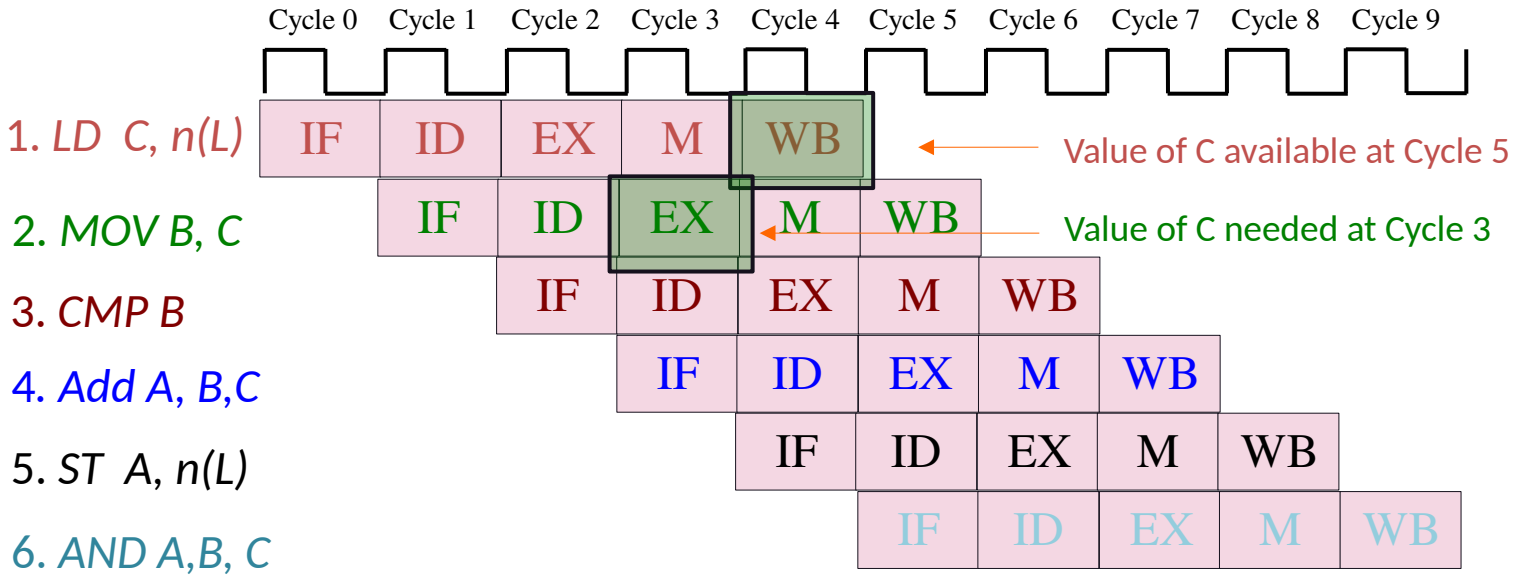
Solution: Use of dual port memory, read write together possible

Problems with pipelining (Hazards)

- Structural hazard
- Data hazard
- Control hazard

CPU Pipelining Hazards

Data hazard: One instruction's input depends on the other instruction



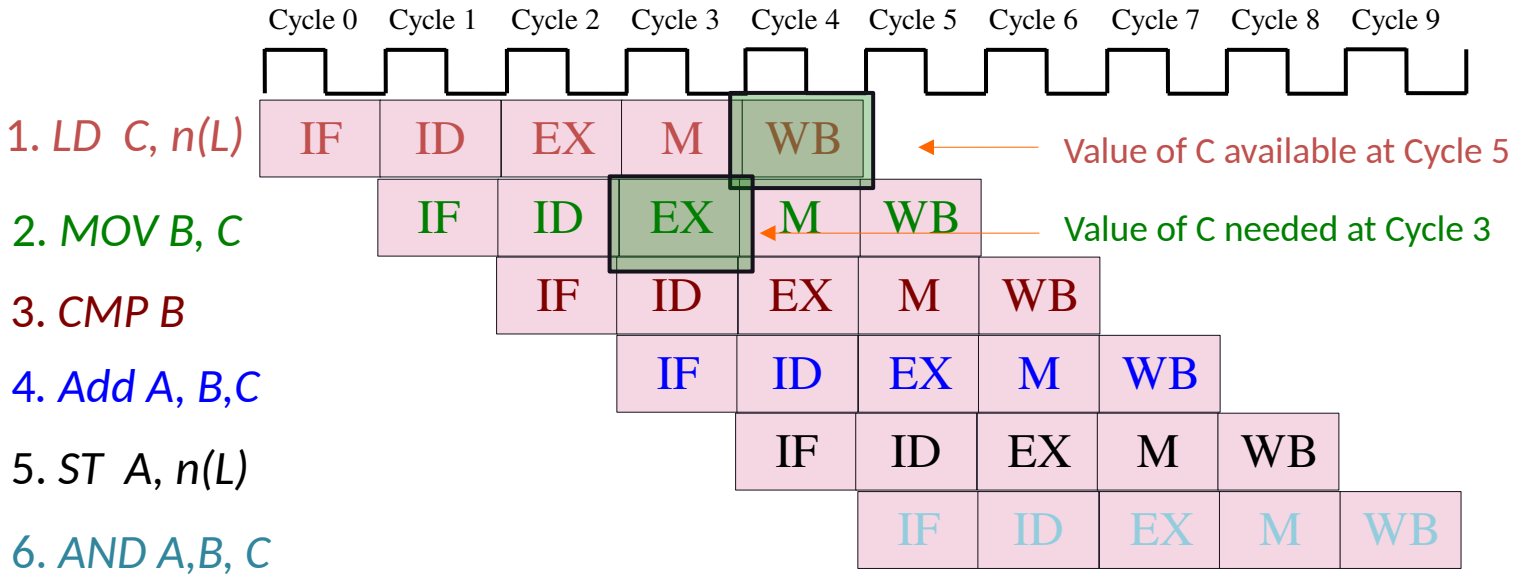
At cycle 5 C is available for MOV operation



Are the any other data hazards?

CPU Pipelining Hazards

Data hazard: One instruction's input depends on the other instruction

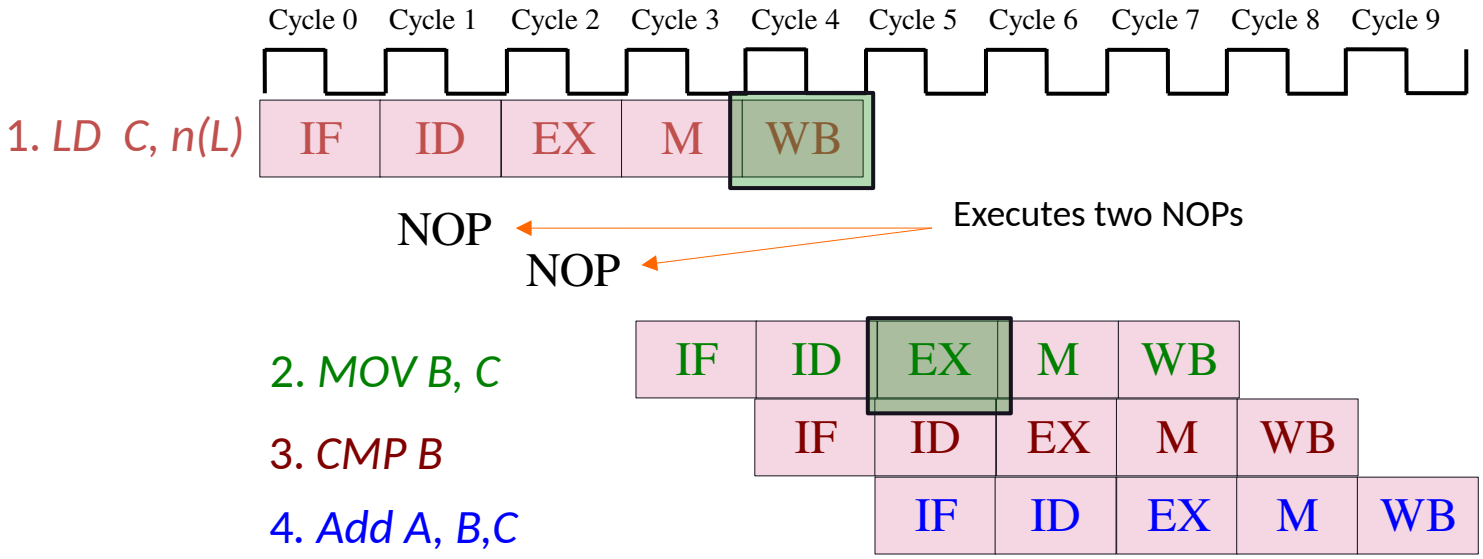


Solutions:

- Software (compiler)
  - NOP
- Hardware
  - Stalling
  - Forwarding

CPU Pipelining Hazards

Data hazard: One instruction's input depends on the other instruction



Solutions:

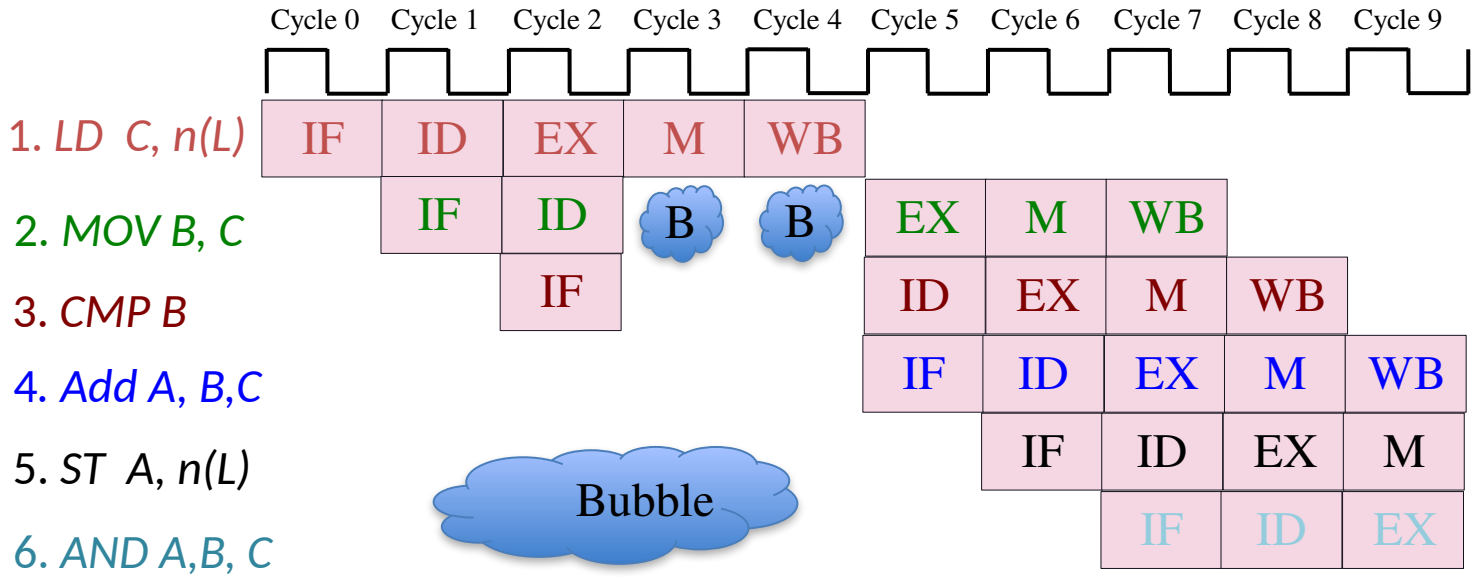
- Software (compiler)
  - NOP
- Hardware
  - Stalling
  - Forwarding

NOP delayed the next instruction execution

Solution relies on compiler

CPU Pipelining Hazards

Data hazard: One instruction's input depends on the other instruction



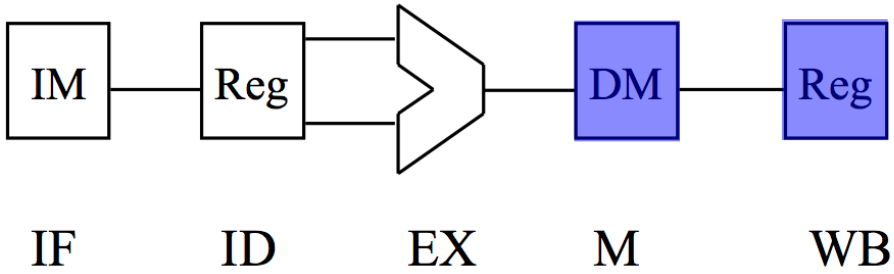
How to implement bubble?

Solutions:

- Software (compiler)
  - NOP
- Hardware
  - Stalling
  - Forwarding

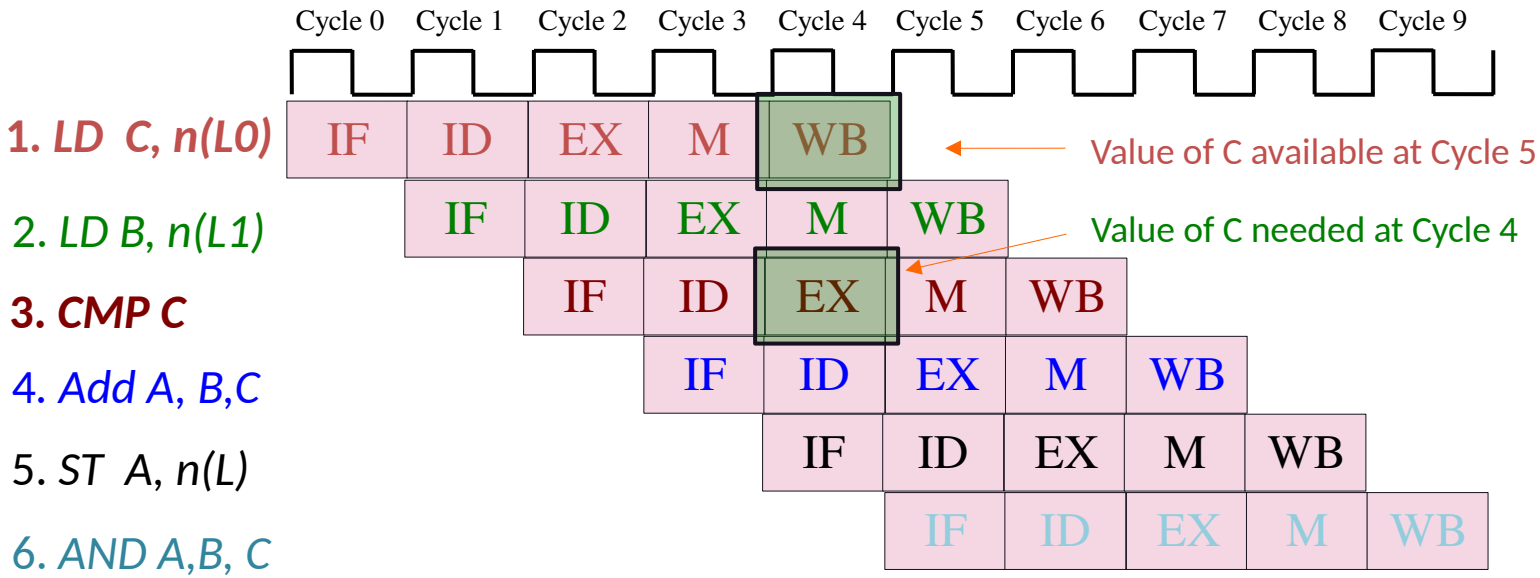
All stages except M and WB will remain inactive at cycles 3 and 4





CPU Pipelining Hazards

Data hazard: One instruction's input depends on the other instruction

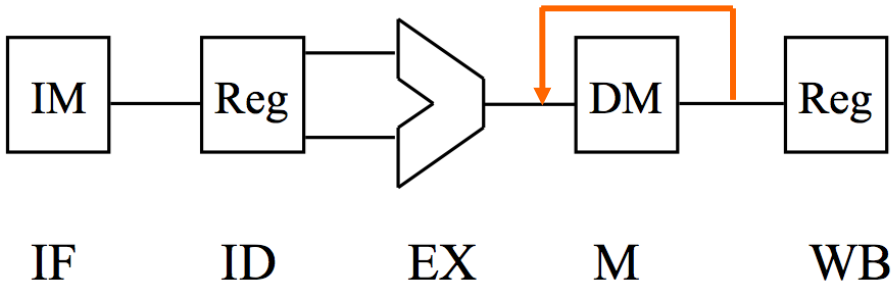


Solutions:

- Software (compiler)
  - NOP
- Hardware
  - Stalling
  - Forwarding

$INS3_EX_R_{in}$ = $INS1_M_R_{out}$

What about Instⁿ 4 and 5?



Problems with pipelining (Hazards)

- Structural hazard
- Data hazard
- Control hazard

CPU Pipelining Hazards

Control hazard: One execution depends on other

L1:7. *MUL A, B,C*

8. *ST A, n(L)*

1. *LD C, n(L)*

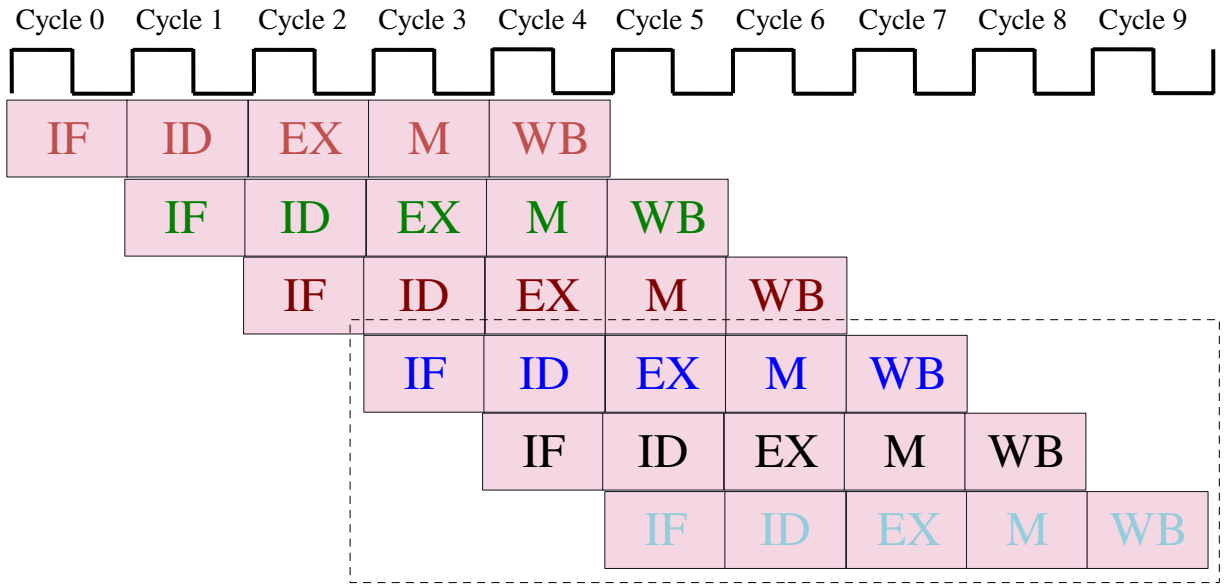
2. *LD B, m(L)*

3. *JE B,C, L1*

4. *Add A, B,C*

5. *ST A, n(L)*

6. *AND A,B, C*



Execution of instructions 4 to 6 depends on branch condition

CPU Pipelining Hazards

Control hazard: One execution depends on other

L1:7. *MUL A, B,C*

8. *ST A, n(L)*

1. *LD C, n(L)*

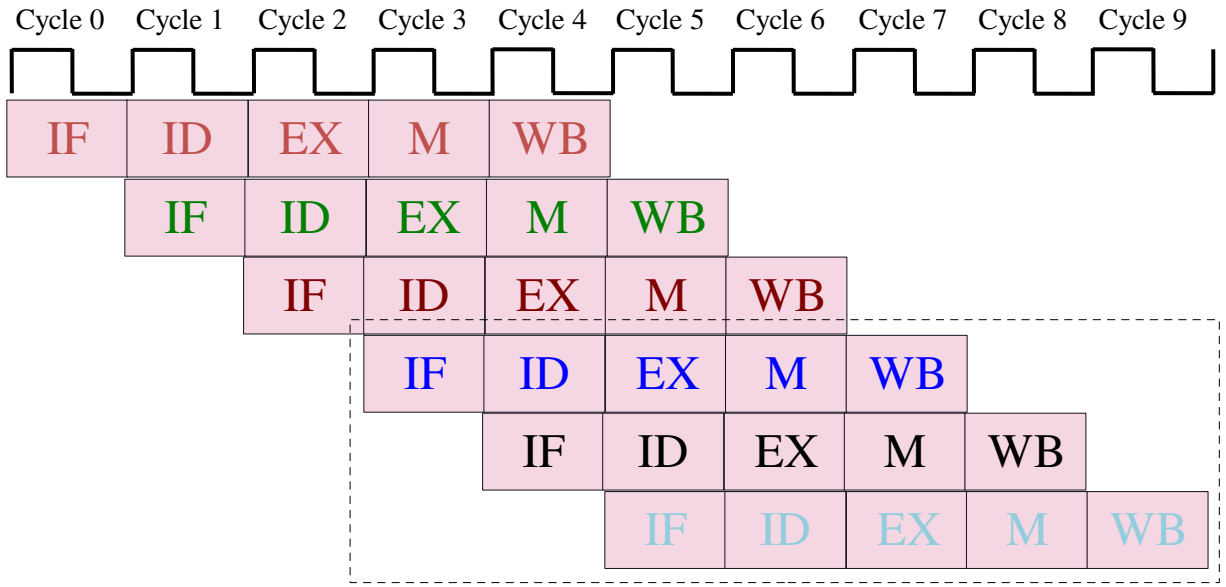
2.LD B, m(L)

3. JE B,C, L1

4. Add A, B,C

5. *ST A, n(L)*

6. AND A,B, C



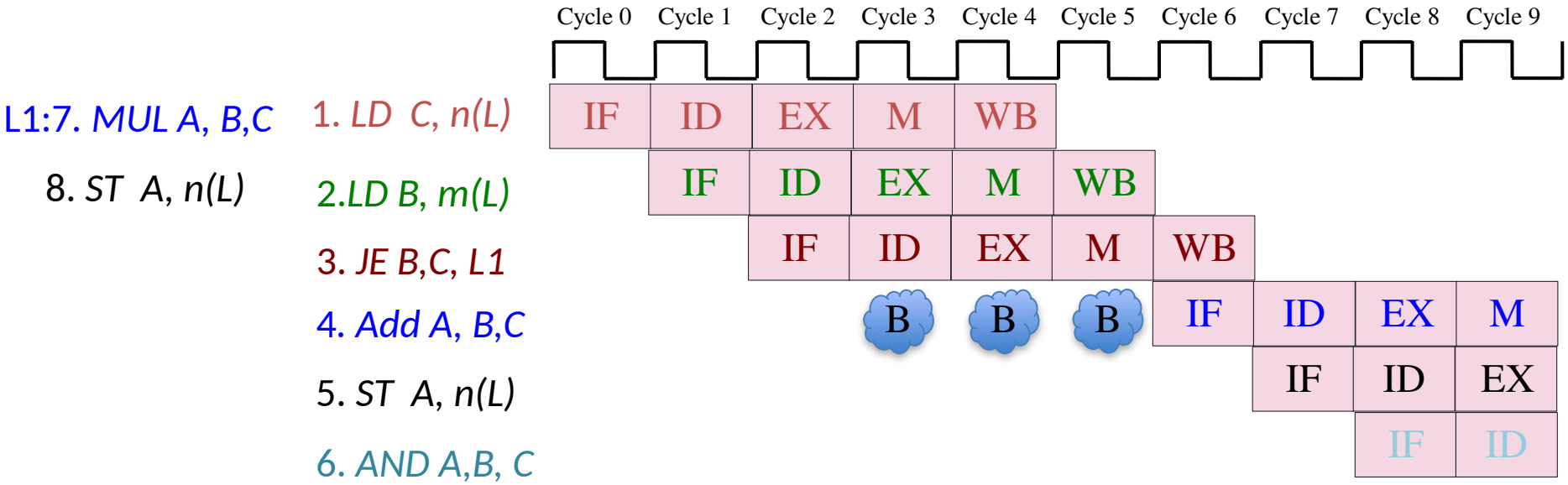
Execution of instructions 4 to 6 depends on branch condition

Solutions:

- Hardware
  - Stalling until condition
  - Early computation of branch condition (branch prediction)
  - Guess which direction
- Software (compiler)
  - NOP

CPU Pipelining Hazards

Control hazard: One execution depends on other



Solutions:

- Hardware
  - Stalling until condition

Cons:

All branches cost 4 cycles

CPU Pipelining Hazards

Control hazard: One execution depends on other

L1:7. *MUL A, B,C*

8. *ST A, n(L)*

1. *LD C, n(L)*

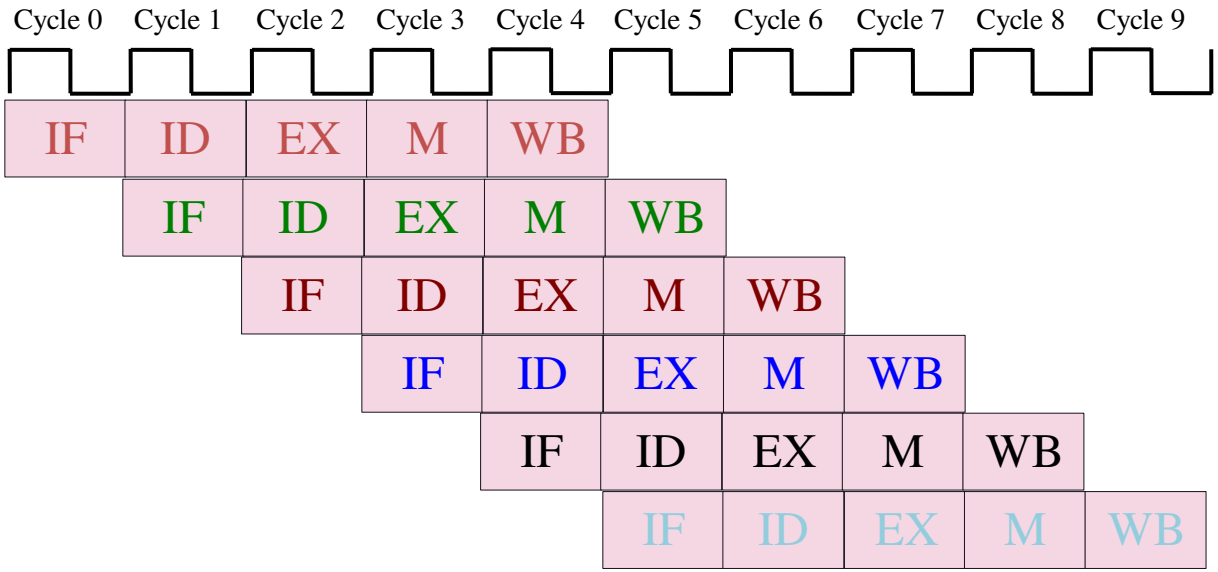
2.LD B, m(L)

3. *JE B,C, L1*

4. Add A, B,C

5. *ST A, n(L)*

6. AND A,B, C



Solutions:

- Hardware
  - Stalling until condition
  - Early computation of branch condition (branch prediction)
  - Guess which direction
- Software (compiler)
  - NOP

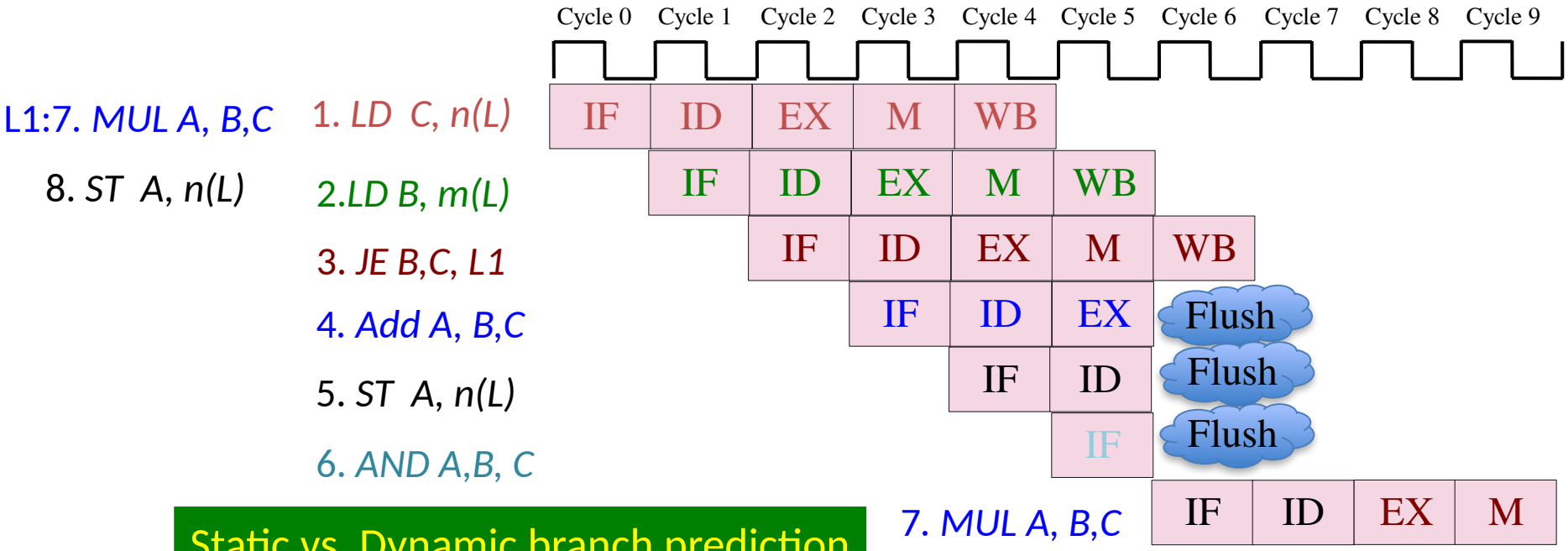
Assume branch will not take place

If (True)

No worries

CPU Pipelining Hazards

Control hazard: One execution depends on other



Static vs. Dynamic branch prediction

Solutions:

- Hardware
  - Stalling until condition
  - Early computation of branch condition (branch prediction)
  - Guess which direction
- Software (compiler)
  - NOP

Assume branch will not take place

If (True)

No worries

Else

Flush pipeline

Feed branch instructions

CPU Pipelining Hazards

Control hazard: One execution depends on other

L1:7. *MUL A, B,C*

8. *ST A, n(L)*

1. *LD C, n(L)*

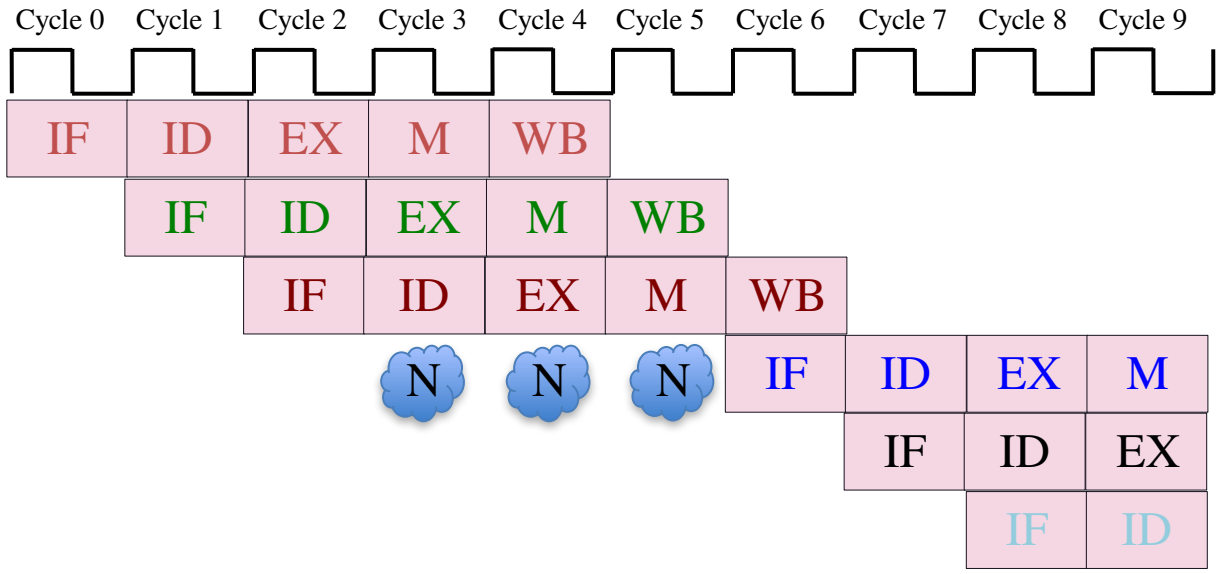
2.LD B, m(L)

3. *JE B,C, L1*

4. Add A, B,C

5. *ST A, n(L)*

6. AND A,B, C



Solutions:

- Hardware
  - Stalling until condition
  - Early computation of branch condition (branch prediction)
  - Guess which direction
- Software (compiler)
  - NOP

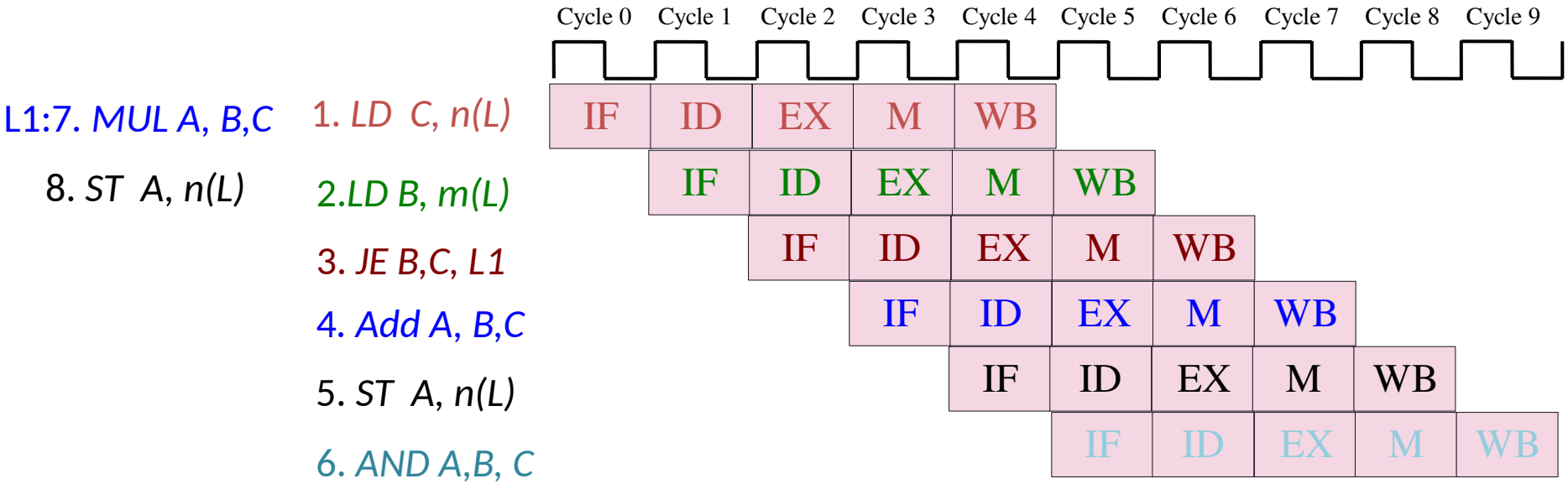
Use NOP until condition

Next Day

- Memory management
- Class test (Wednesday 4.15PM)

CPU Pipelining Hazards

Control hazard: One execution depends on other



Solutions:

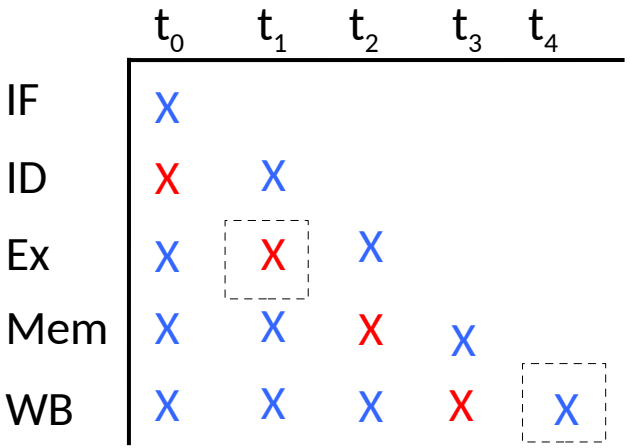
- Hardware
  - Stalling until condition
  - Early computation of branch condition (branch prediction)
  - Guess which direction
- Software (compiler)
  - NOP

Use NOP until condition

CPU Pipelining Hazards

Data hazard: One instruction input depend on the other instruction

Ex. Read and Write *Add A, B,C* *Add D, D,A*

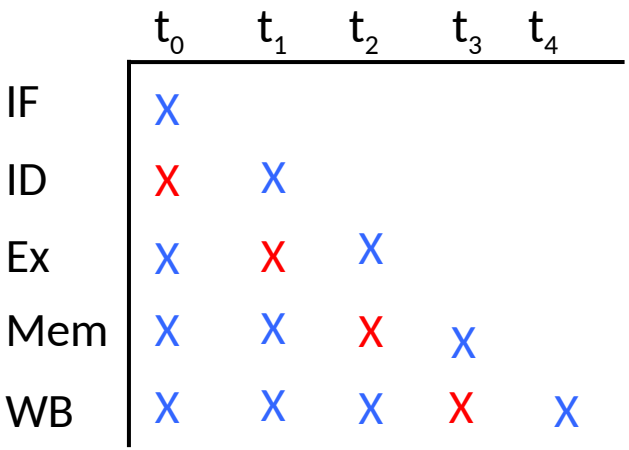


Data in A will be available in t4

CPU Pipelining Hazards

Control hazard: Decision of one instruction execution depend on other

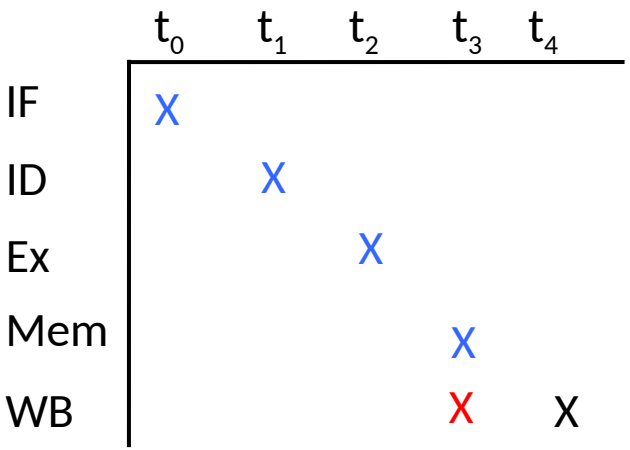
Ex. Branching *JEQ A, B,C*



CPU Pipelining Hazards

Control hazard: One execution depends on other

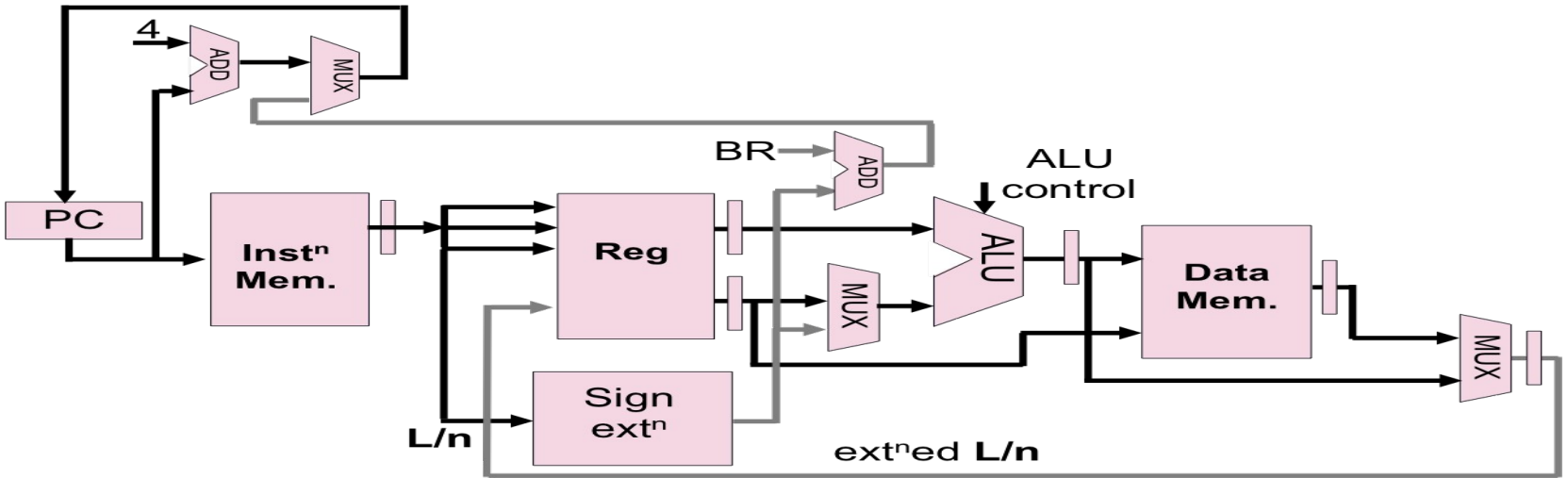
Ex. Single memory *ST A, L* *Add A, B,C*



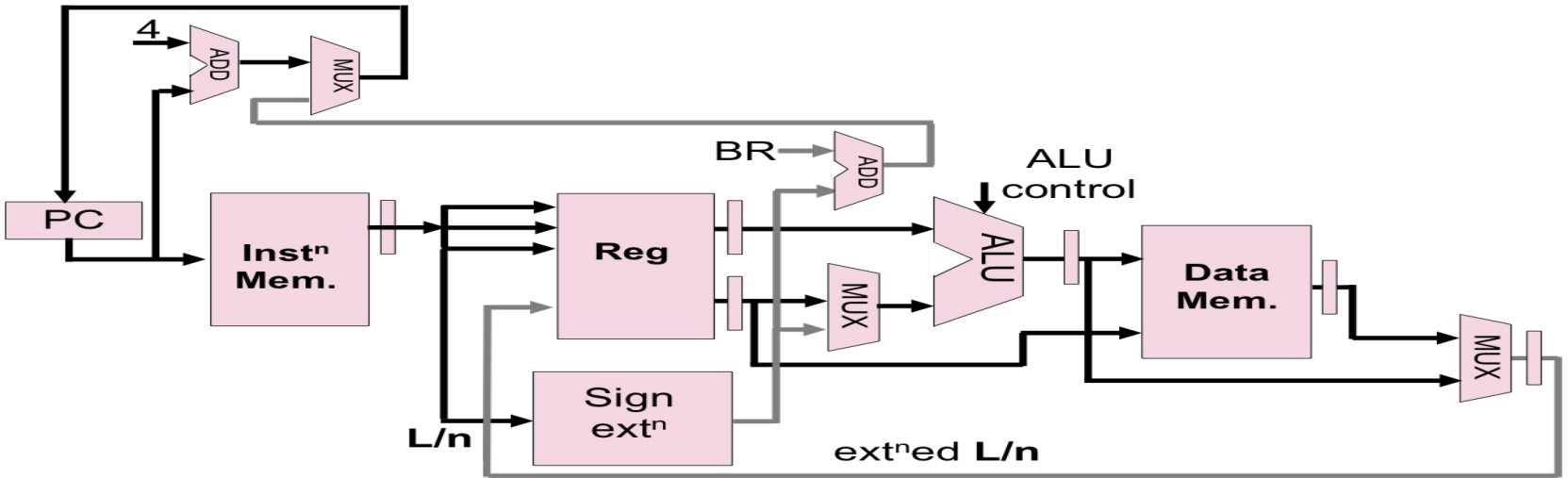
Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*
6. *AND A,B, C*
7. *CMP A*

	t_0	t_1	t_2	t_3	t_4		
IF	X	X	X	X	X	X	X
ID		X	X	X	X	X	X
Ex			X	X	X	X	X
Mem				X	X	X	X
WB					X	X	X



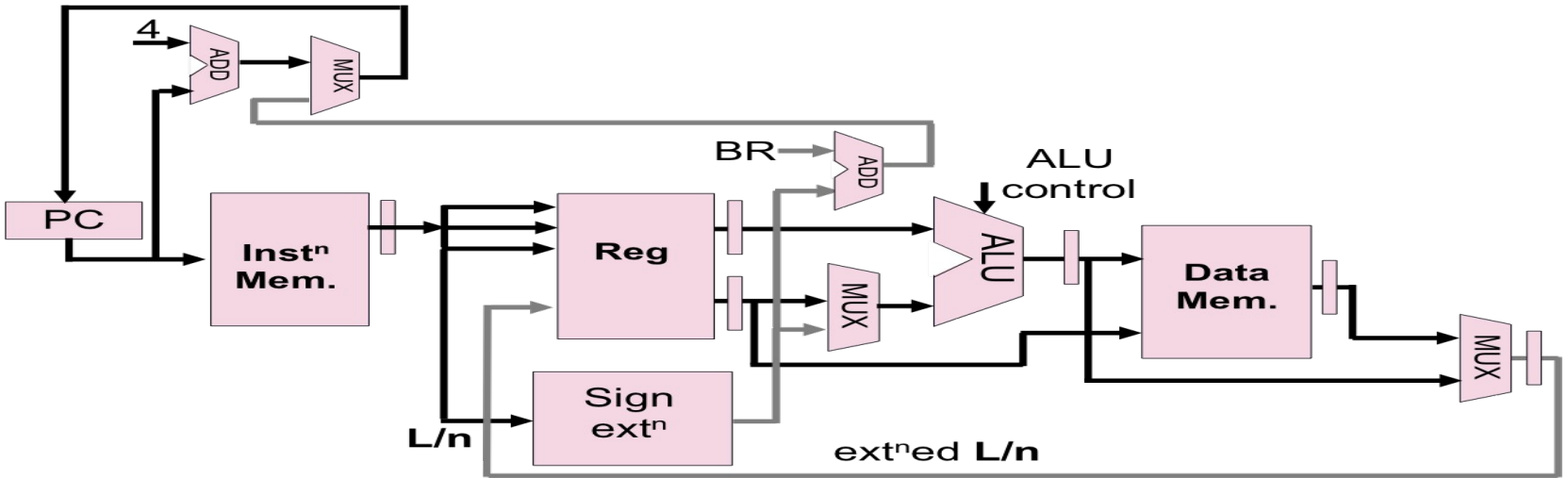
Instruction Pipelining



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*

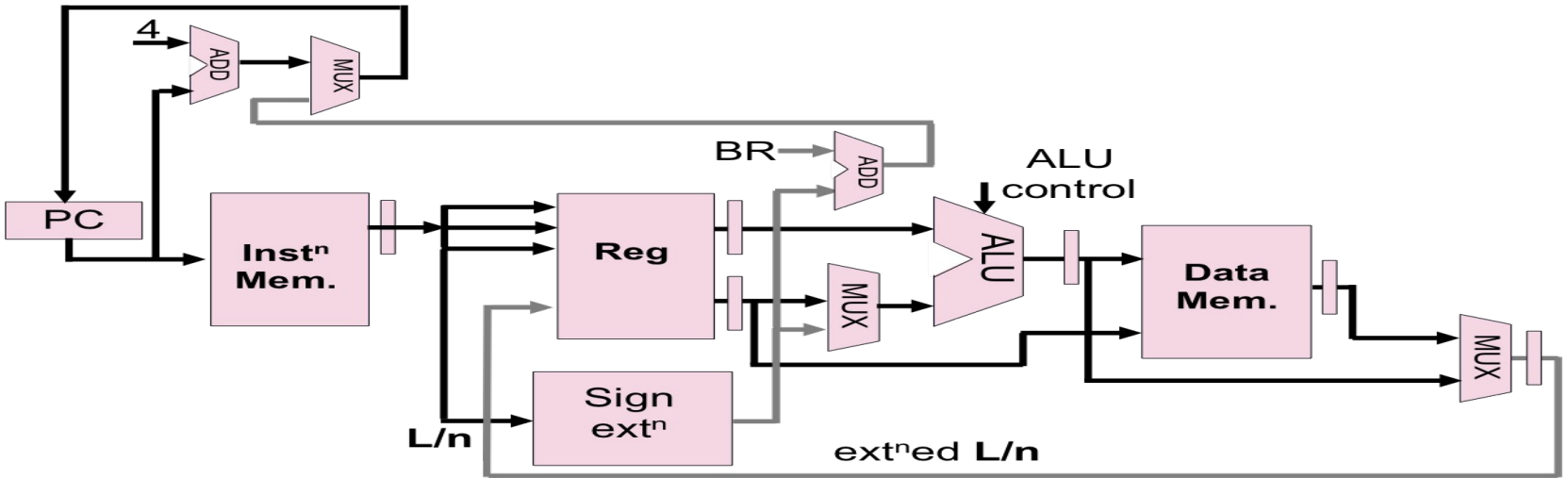
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X					
ID		X					
Ex							
Mem							
WB							



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*

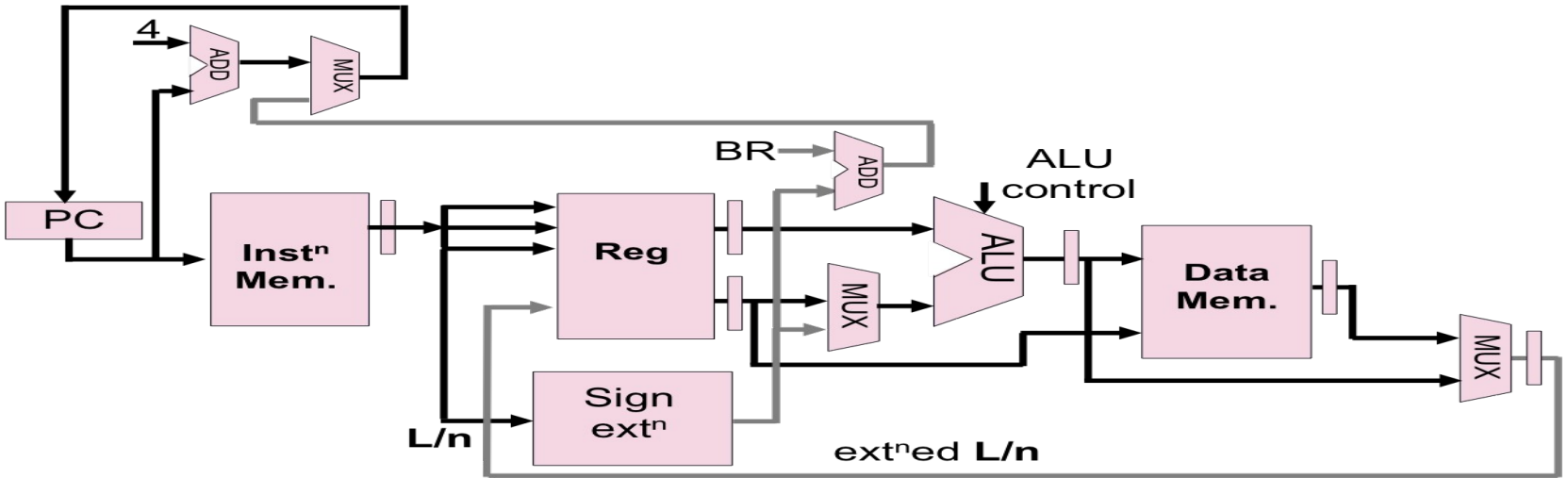
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X				
ID		X	X				
Ex			X				
Mem							
WB							



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*

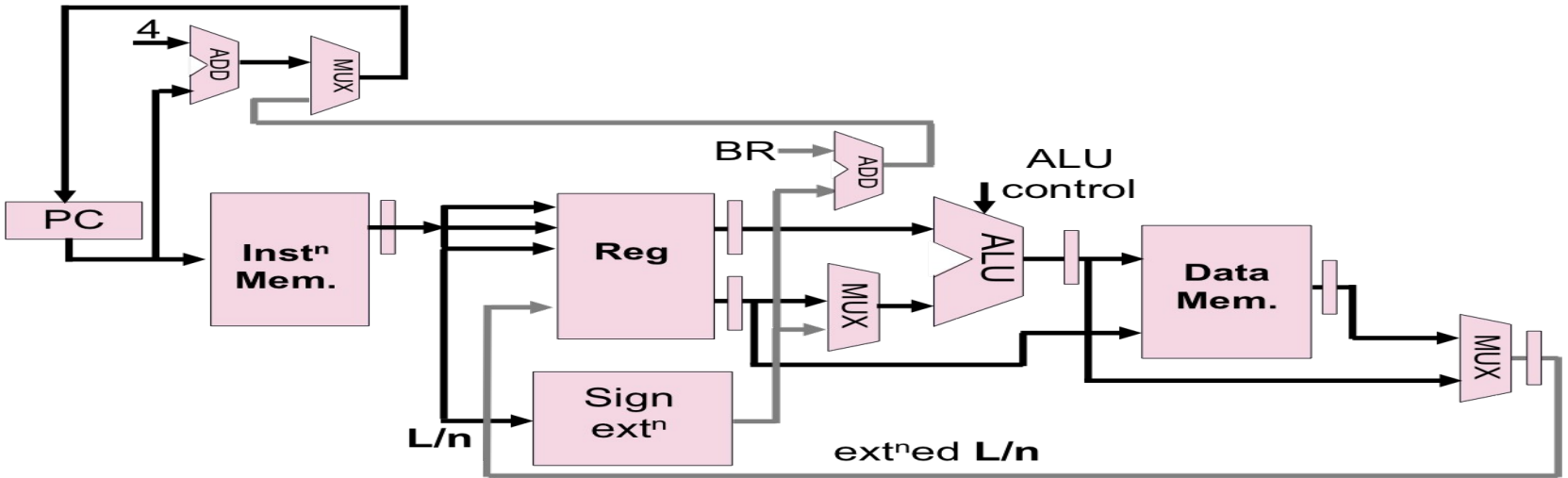
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X			
ID		X	X	X			
Ex			X	X			
Mem				X			
WB							



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*

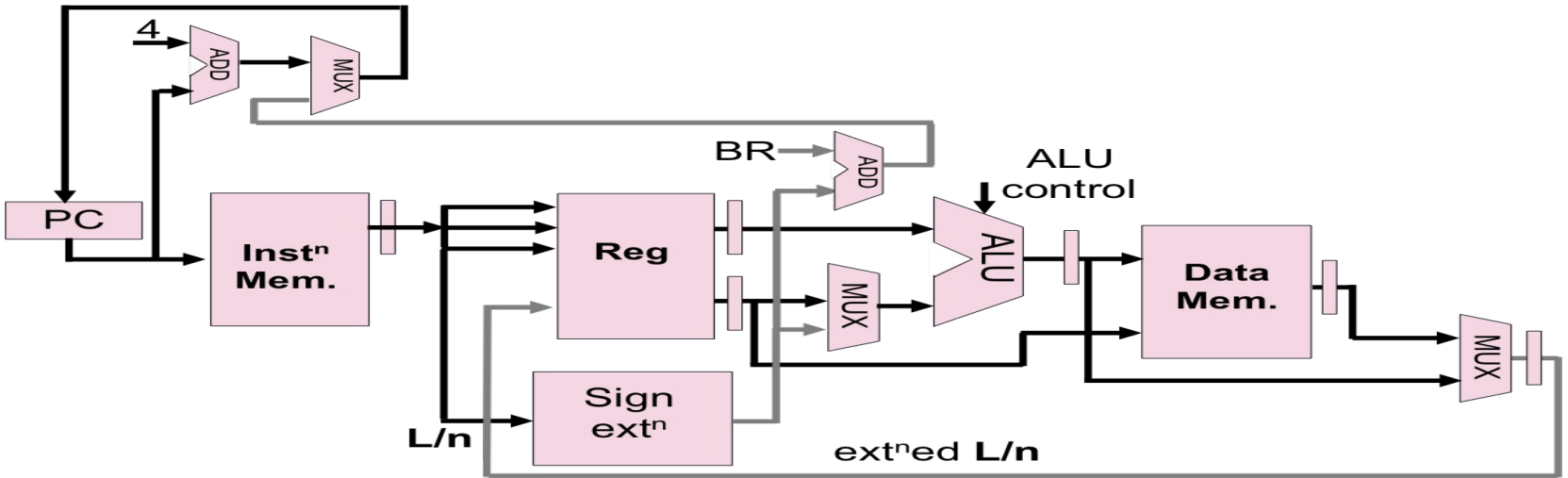
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X	X		
ID		X	X	X	X		
Ex			X	X	X		
Mem				X	X		
WB					X		



Instruction Pipelining

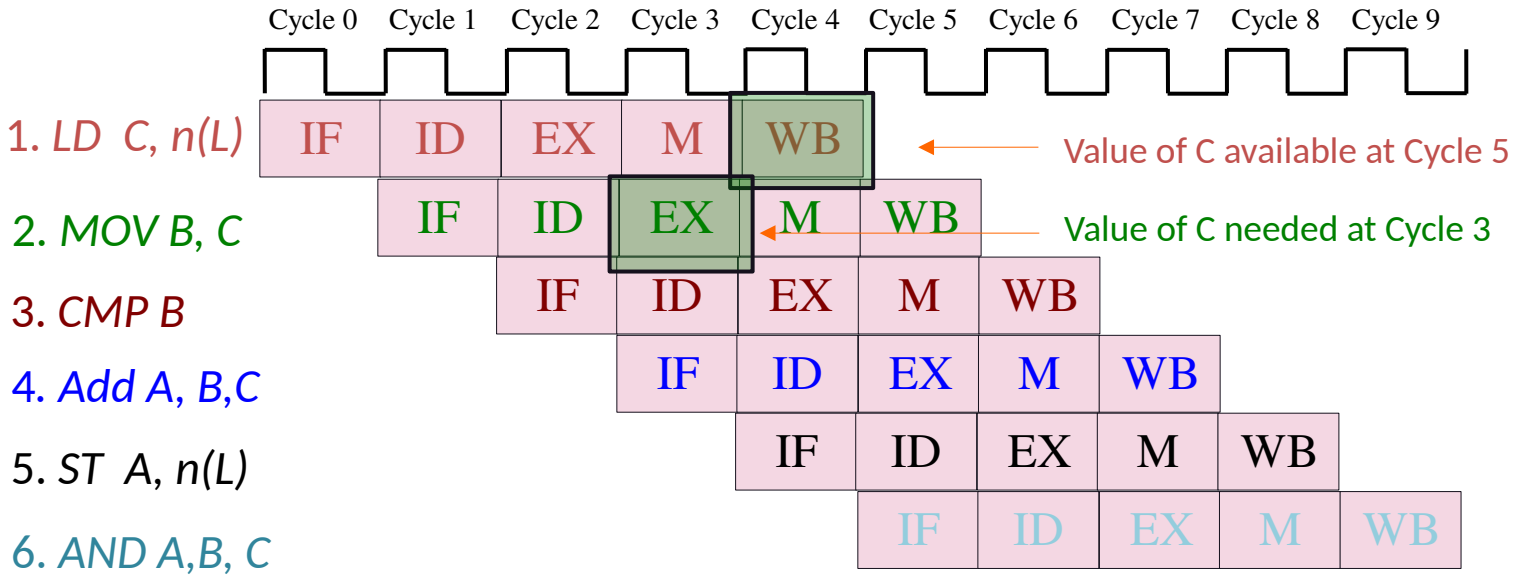
1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*
6. *AND A,B, C*

	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X	X	X	
ID		X	X	X	X	X	
Ex			X	X	X	X	
Mem				X	X	X	
WB					X	X	



CPU Pipelining Hazards

Data hazard: One instruction's input depends on the other instruction

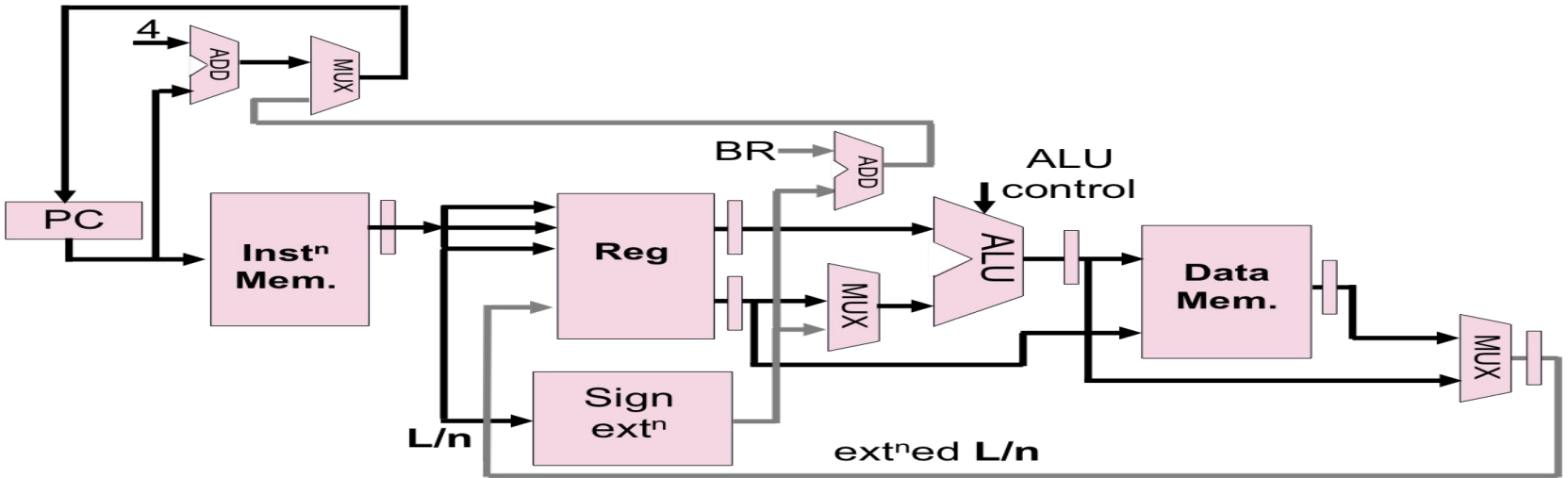


Solution: Use of dual port memory, read write together possible

Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*
6. *AND A,B, C*
7. *CMP A*

	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X	X	X	X
ID		X	X	X	X	X	X
Ex			X	X	X	X	X
Mem				X	X	X	X
WB					X	X	X



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*
6. *AND A,B, C*
7. *CMP A*

	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X	X	X	X
ID		X	X	X	X	X	X
Ex			X	X	X	X	X
Mem				X	X	X	X
WB					X	X	X

